

Read-Me Document

Login for User and Admin Example

user login:

- 👤 Username: amy455
- 👤 Password: justforfun
- Admin key (blue copyright symbol): TECSW3326^

Comments about our app:

- We ran our app on a Pixel 8 API 37.0

Uncompleted Functionality

- We did not complete the search functionality on the home screen of our app
- Also didn't have other conditions like price and condition to filter by · Sellers also cannot edit or delete their own listings as of right now.

On the admin dashboard,

- "Database Items" shows all pending, rejected, and approved items
- "Approved Items" card doesn't change to another screen because admin can view approved items from the "Dashboard Items" card

The Realtime Database in Firebase, stores the admin key, Banner photo, and Category IDs, and all the items, pending, approved and rejected.

- The pictures were not stored in Firebase but were put into Cloudinary and the picture link from Cloudinary was put into the Realtime database.

Project: TechSwap
Team No.: 9
Class: CSE 3310; Fall 2026 Module: System Requirements Analysis (SRA) Deliverable: SRA Document

Version: [4.0] Date: [04/23/2026]

Revision History

Version number	Date	Originator	Reason for change	High level description of changes
1.0	3/2/2026	Team 9	Initial Draft	Create the SRA document with an overview of the project's outline and diagrams.
2.0	3/31/2026	Team 9	Added Test Plan to increment 2 (SRA Document)	Included a detailed test plan outlining testing procedures based on requirements for increment 2.
3.0	4/20/2026	Team 9	1. Edited the description for filter 2. Edited the description for some system requirements 3. Edited the Test Plan	1. Changed it to filter by category rather than price or condition 2. Edited existing seller listing description 3. Edited shopping cart description 4. Revised the Test plan
4.0	4/23/2026	Team 9	Added screenshots 5.4.1-5.4.16 Added Main Code	Added screenshots of the screen design. Added the main code screenshots.

Table of Contents

Read-Me Document.....	1
Revision History.....	3
Table of Contents.....	2
1. Introduction and Project Overview.....	4
2. Objectives.....	6
2.1 BUSINESS OBJECTIVES.....	6
2.2 SYSTEM OBJECTIVES.....	7
3. PROJECT CONTEXT DIAGRAM.....	8
4. Systems Requirements.....	9
4.1 “LOGIN” REQUIREMENTS.....	9
4.2 “REGISTRATION” REQUIREMENTS.....	9
4.3 “ELECTRONIC LISTINGS” REQUIREMENTS.....	10
4.4 “SEARCH & FILTER” REQUIREMENTS.....	10
4.5 “SELLER LISTING MANAGEMENT” REQUIREMENTS.....	11
4.6 “ADMIN DASHBOARD” REQUIREMENTS.....	11
4.7 “SHOPPING CART” REQUIREMENTS.....	12
4.8 “CHECKOUT/PAYMENT” REQUIREMENTS.....	12
4.9 “ORDER CONFIRMATION” REQUIREMENTS.....	13
5. Software Processes and Infrastructure.....	14
5.1 HARDWARE AND INFRASTRUCTURE.....	14
5.2 UML DIAGRAMS.....	15
5.2.1 MAIN APP MODEL.....	15
5.2.2 STD – LOGIN AND REGISTRATION.....	16
5.2.3 SEQUENCE – ADMIN AND LISTINGS.....	17
5.2.4 STD – SELLER AND LISTINGS.....	18
5.2.5 ACTIVITY – LISTING LIFECYCLE.....	19
5.2.6 ACTIVITY – SEARCH AND FILTER.....	20
5.2.7 CLASS –CART COMPONENTS.....	21
5.2.8 SEQUENCE – CART AND CHECKOUT.....	22
5.3 TEST PLAN.....	23
5.3.1 INTRODUCTION AND PLAN OF APPROACH.....	23
5.3.2 TEST CASES.....	24
5.3.2.1 - REGISTRATION.....	24
5.3.2.2 - LOGIN.....	25
5.3.2.3 – ELECTRONIC LISTINGS.....	26
5.3.2.4 SEARCH AND FILTER.....	27
5.3.2.5 – SELLER LISTING MANAGEMENT.....	28
5.3.2.6 – ADMIN DASHBOARD.....	29
5.3.2.7 SHOPPING CART.....	30
5.3.2.8 CHECKOUT/PAYMENT.....	31
5.3.2.9 ORDER CONFIRMATION.....	32
5.4 USER MODEL.....	33
5.4.1 SPLASH SCREEN.....	33

5.4.2 LOGIN SCREEN.....	34
5.4.3 REGISTER SCREEN.....	35
5.4.4 HOME PAGE SCREEN.....	36
5.4.5 FILTER BY CATEGORY SCREEN.....	37
5.4.6 ITEM DETAILS SCREEN.....	38
5.4.7 SELLER DASHBOARD SCREEN.....	39
5.4.8 CREATE A LISTING SCREEN.....	40
5.4.9 PROFILE PAGE SCREEN.....	41
5.4.10 CART SCREEN.....	42
5.4.11 CHECKOUT PAGE SCREEN.....	43
5.4.12 ORDER CONFIRMATION SCREEN.....	44
5.4.13 ADMIN DASHBOARD SCREEN.....	45
5.4.14 VIEW ALL LISTINGS SCREEN.....	46
5.4.15 VIEW PENDING ITEMS SCREEN.....	47
5.4.16 VIEW ALL USERS SCREEN.....	48
5.5 MAIN SELECTED SOURCE CODE.....	49
5.5.1 LOGIN ACTIVITY CODE.....	49
5.5.2 SELLER LISTINGS ACTIVITY CODE.....	57
5.5.3 ADMIN ACTIVITY CODE.....	64
5.6 DATABASE TABLES.....	68
5.6.1 USER DATABASE: FIRESTORE.....	68
5.6.2 ITEMS AND CATEGORY: REALTIME DATABASE.....	69
6. Assumptions and Constraints.....	70
6.1 ASSUMPTIONS.....	70
6.2 CONSTRAINTS.....	70
6.3 OUT OF SCOPE MATERIAL.....	70
7. Delivery and Schedule.....	71
8. Stakeholder Approval Form.....	72
Appendix:.....	73

1. Introduction and Project Overview

TechSwap is an electronic marketplace application designed specifically for students who want to buy or sell electronics in a secure and efficient environment. The platform supports both pre-owned and brand-new electronic products, including phones, laptops, and tablets. TechSwap addresses the problem of unreliable and unsafe peer or peer transactions by providing a centralized marketplace where users can interact safely. The primary problem TechSwap solves is the lack of a safe and organized platform to avoid scams, misleading listings, and poor trading experience for students.

The application supports three primary user roles: buyers, sellers, and administrators. Users are able to switch between buyer and seller roles, allowing flexibility within a single account. Buyers can browse electronic listings, search for specific products, apply filters to narrow results, and purchase electronics through a checkout process. Sellers are responsible for creating and managing electronic listings, while administrators oversee the platform by reviewing users and listings and removing content that is against guidelines.

Overall, TechSwap covers a range of core functional areas necessary to support the student electronics marketplace. The authentication and account management features allow users to securely create accounts, log in, and maintain their profiles. The platform's listing management system gives sellers the ability to post, edit, and remove electronics from the marketplace, while buyers are able to browse and filter listings by category, condition, and price range, add items to their cart, and complete the checkout process. The administrative features allow administrators to oversee listings posted on the platform, manage user accounts, and ensure that all content meets community guidelines.

Major System Features

User Registration: New users can create an account by providing required personal information and selecting an appropriate system of role.

Secure Authentication: Registered users can log in to the system using validated credentials to access authorized features.

Role Based Access Control: The system assigns permissions based on user roles, regulates access to listings, and administrative functions.

Listing Management: Authorized sellers can create, update, and remove electronic product listings.

Product Search: Buyers can search for electronics by their name to save time from scrolling through the other listings.

Filtering: Refine results using categories and filter options based on new or used items.

Order Processing: Provides confirmation for products that have successfully been purchased.

Order History: Users can view records of previous orders and transactions.

Profile Management: Users can view and update personal account information.

Shopping Cart Management: Buyers can select products and manage them in a cart prior to ordering and submitting.

2. Objectives

2.1 BUSINESS OBJECTIVES

Login page will help users login to their accounts:

- Username
- Password
- Forgot Password
- Forgot Username

Registration will store the users personal information:

- First Name
- Last Name
- E-mail address
- Username
- Password
- Security Question(s)

Electronics Listings for buyers to rent:

- Phones
- Laptops
- Tablets

Search help locate specific information from the listings:

- Keyword search bar

Filter helps filter specific information from the listings:

- Filter by category

Seller Listing Management provide sellers with tools to:

- Create new listing
- Add pricing, description, and condition

Admin Dashboard allow administrator to:

- Review and approve seller accounts
- Monitor user activity
- Moderate and remover inappropriate listings or users

Shopping Cart will allow buyers to manage items before checkout:

- Add item to cart
- View cart items
- Remove item from cart
- Update item quantity
- View item price

- View total cost of all items
- Save item for later
- Clear entire cart
- Proceed to checkout
- Return to product listings

Checkout/Payment for the buyer to make a payment:

- Review order summary
- Enter mock payment details (card number, name, expiration date)
- Select payment method (Credit/Debit)
- Confirm billing information
- Cancel checkout
- Place order

Order Confirmation provides a confirmation after checkout:

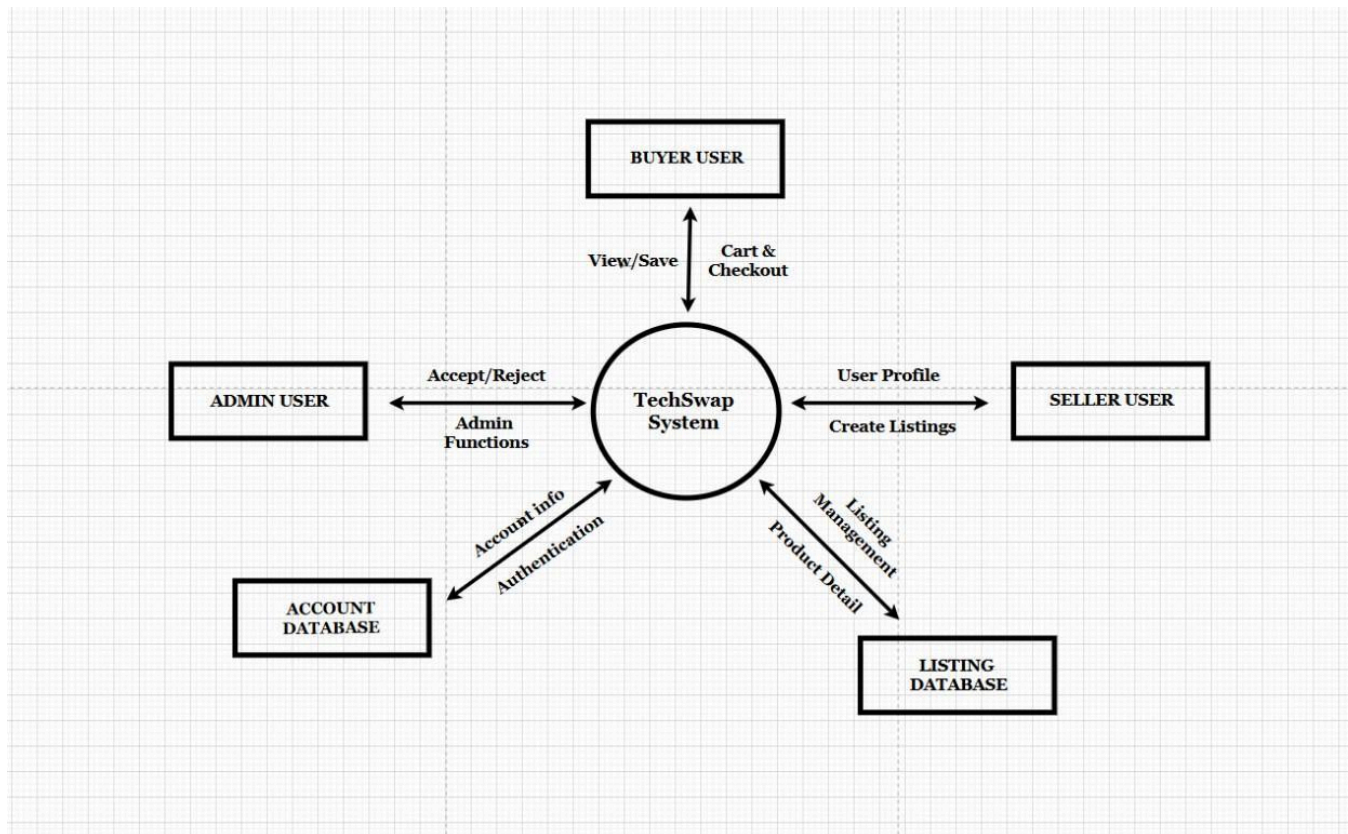
- View order confirmation number
- View purchased items
- View total amount paid
- View order date and time
- Return to home page

2.2 SYSTEM OBJECTIVES

The goal of the system is:

- Provide a login/create an account page.
- Store user profiles and transactions using Firebase.
- Enable search and filter functionality for listings.
- Enforce role-based access control (Buyer, Seller, Admin)
- Support CRUD operations (Create, Read, Update, Delete) for listings.
- Provide a clear and user-friendly interface with smooth navigation.
- Provide a secure marketplace app for students.

3. PROJECT CONTEXT DIAGRAM



4. Systems Requirements

4.1 “LOGIN” REQUIREMENTS

Requirement Title:	• User log in and system authentication
Sequence No:	• 1
Short description:	• Checks if the user’s login information is correct and grants access to their account.
Description:	• Authenticates users by verifying entered email/username and password against stored credentials. • Provides “Forgot Password” and “Forgot Username” options for account recovery.
Pre-Conditions:	• User is registered in the system. • User is not currently logged in.
Post Conditions:	• User is logged in successfully. • User gains access to their TechSwap account with the correct role (Buyer, Seller, or Admin).
Other attributes:	• Passwords will be stored using secure algorithms.

4.2 “REGISTRATION” REQUIREMENTS

Requirement Title:	• Registration
Sequence No:	• 2 (from Login/Role Selection)
Short description:	• Allows a new user to create an account and select a role.
Description:	• Collects and stores user’s personal information (first name, last name, email, username, password, security question(s), and chosen role: Buyer, Seller, or Admin) in Firebase. • Flags seller accounts for admin verification before posting listings.
Pre-Conditions:	• User is not registered • Users are not logged in.
Post Conditions:	• A new user account has been created. • Seller accounts are marked as “pending verification” awaiting admin approval.
Other attributes:	• N/A

4.3 “ELECTRONIC LISTINGS” REQUIREMENTS

Requirement Title:	• Electronic Listings
Sequence No:	• 3 (from Homepage/Search)
Short description:	• Displays available for students to browse and rent/buy.
Description:	• Retrieves and displays electronics listing (phones, laptops, tablets, keyboards, etc.) with details such as photos, product name, brand/model, price, condition, and availability. • Allows buyers to view and select items.
Pre-Conditions:	• User is logged in as a Buyer or Seller. • Listings exist in the system.
Post Conditions:	• User views one or more electronics listings. • Users can select products to view their details.
Other attributes:	• N/A

4.4 “SEARCH & FILTER” REQUIREMENTS

Requirement Title:	• Search & Filter
Sequence No:	• 4 (from Electronics Listings/Homepage)
Short description:	• Let users search and narrow listings by keywords and filters.
Description:	• Allows keyboard search by product name/model. • Enables optional filtering by category, condition, and price range. • Displays a filtered list of matching electronics within the listing screen.
Pre-Conditions:	• User is on the listings or homepage screen. • Products are loaded and available.
Post Conditions:	• Search results matching the keyboard, and filters are displayed.
Other attributes:	• N/A

4.5 “SELLER LISTING MANAGEMENT” REQUIREMENTS

Requirement Title:	• Seller Listing Management
Sequence No:	• 5 (from Seller mode / My Listings)
Short description:	• Allows sellers to create and manage their electronics listings.
Description:	• Let's sellers create new listings or edit existing ones. • Enables updating product name, brand/model, category, price, condition, description, photos, and availability status. • Saves changes in Firebase. • Reflects updates to buyers after admin approval if required.
Pre-Conditions:	• User is logged in as a Seller • The seller account has been verified by an Admin.
Post Conditions:	• New or updated listings are stored in Firebase. • Listing shows updated details to buyers. Price, description, availability
Other attributes:	• N/A

4.6 “ADMIN DASHBOARD” REQUIREMENTS

Requirement Title:	• Admin Dashboard
Sequence No:	• 6 (from Admin Login)
Short description:	• Provides tools for admins to manage users and listings.
Description:	• Displays basic statistics such as user and listing counts. • Provides tabs to view user details and listing details. • Allows admins to approve or disable seller accounts.
Pre-Conditions:	• User is logged in with the admin role.
Post Conditions:	• User listing statuses may be updated (approved, disabled, or removed) • Changes are stored in Firebase.
Other attributes:	• N/A

4.7 “SHOPPING CART” REQUIREMENTS

Requirement Title:	• Shopping Cart
Sequence No:	• 7 (from Product Details)
Short description:	• Allows buyers to manage selected items before checkout.
Description:	• Let's buyers add items to cart and view contents. • Enables removing items, updating quantities, viewing items and total prices. • Allows saving items for later or clearing the cart. • Provides options to proceed to checkout or return to listings.
Pre-Conditions:	• User is logged in as a Buyer. • Buyer has selected at least one product.
Post Conditions:	• Cart state (items, quantities, saved items) is updated and stored. • User may navigate to the checkout or back to listings.
Other attributes:	• N/A

4.8 “CHECKOUT/PAYMENT” REQUIREMENTS

Requirement Title:	• Checkout/Payment
Sequence No:	• 8 (for Shopping Cart)
Short description:	• Processes the buyer's mock payment and order details.
Description:	• Shows order summary and collects shipping and payment information. • Includes card details, name, expiration date, and payment method. • Let's buyers confirm billing info, cancel checkout, or place order. • Creates an order record with status and total amount upon confirmation.
Pre-Conditions:	• User is logged in as a Buyer. • The user has at least one item in the cart.
Post Conditions:	• If canceled, no order is created and the user returns to the cart. • If confirmed, a new order is stored in Firebase, and the cart is cleared.
Other attributes:	• N/A

4.9 “ORDER CONFIRMATION” REQUIREMENTS

Requirement Title:	• Order Confirmation
Sequence No:	• 9 (from Checkout/Payment)
Short description:	• Show confirmation after a successful order.
Description:	• Displays order confirmation number, purchased items, total amount paid, and order date/time. • Provides options to return to homepage or continue shopping.
Pre-Conditions:	• Checkout successfully completed • Order record created.
Post Conditions:	• User reviews order details. • User is redirected to the homepage or product listings.
Other attributes:	• N/A

5. Software Processes and Infrastructure

Software Process

Our team will follow a consistent development process and build the application in structured phases to make sure there's steady progress throughout the project, continuing to test the project once each iteration is finished. The development will be divided into 3 parts

Part 1 – Authentication & Core Setup: User registration, login/logout, role selection (Buyer, seller, admin) and basic navigation throughout the app.

Part 2 – Core Features: Electronic listings, search & filter, product details, seller listing management (add, edit, delete), along with the admin dashboard.

Part 3 – Buyer Flow & Polish: Cart, checkout/ payment process, order confirmations, profile settings, and a final look over the application to make sure everything is running smoothly

Each testing phase will be going through thorough testing to make sure no small detail goes unnoticed. The team will meet weekly to make sure everything gets done in time.

Our team will use GitHub for version control to keep our workflow organized and consistent. Each new feature will be developed on its own separate branch before being added to the main application. Once a feature has been built and tested on its branch, it will be reviewed by the team. If it passes testing, it will be merged into the main branch. If it fails, the team will debug and revise the code until it works correctly. No code will be merged into the main branch without first passing the testing phase. Any requirement updates will be documented in the repository to maintain a clear record of changes throughout development.

5.1 HARDWARE AND INFRASTRUCTURE

Android version: Android 14 (API 34 "UpsideDownCake")

Android development environment: Android Studio with Android Virtual Device (AVD) Manager, using a Pixel 8 emulator (1080 x 2400 resolution, 420 dpi, API Level 34, x86_64 system image)

Programming Languages

- Kotlin, XML

Database & IDE

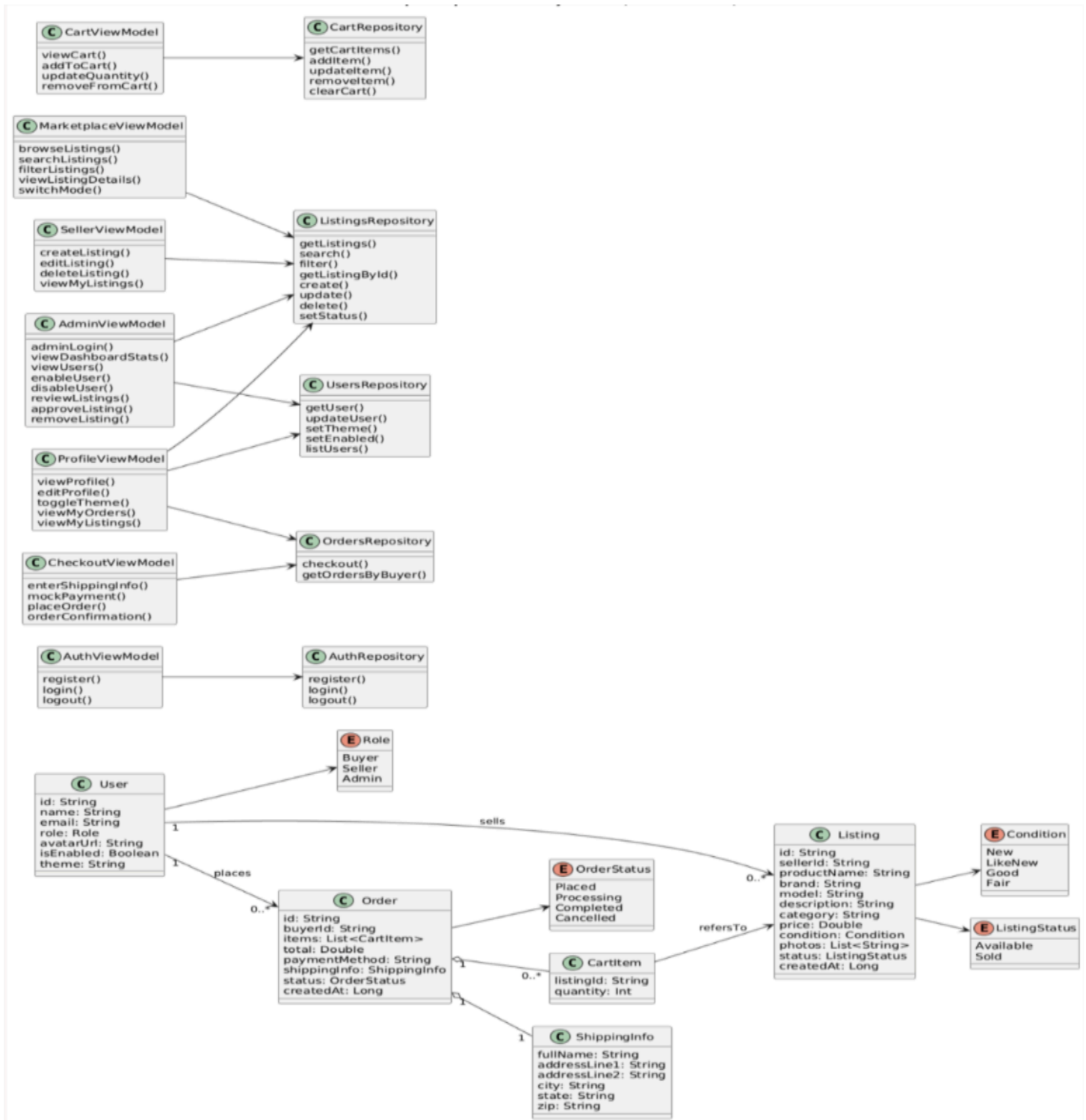
- Android Studio, Firebase Firestore, Firebase Authentication

Documentation Tools

- GitHub, Google Docs, KDoc

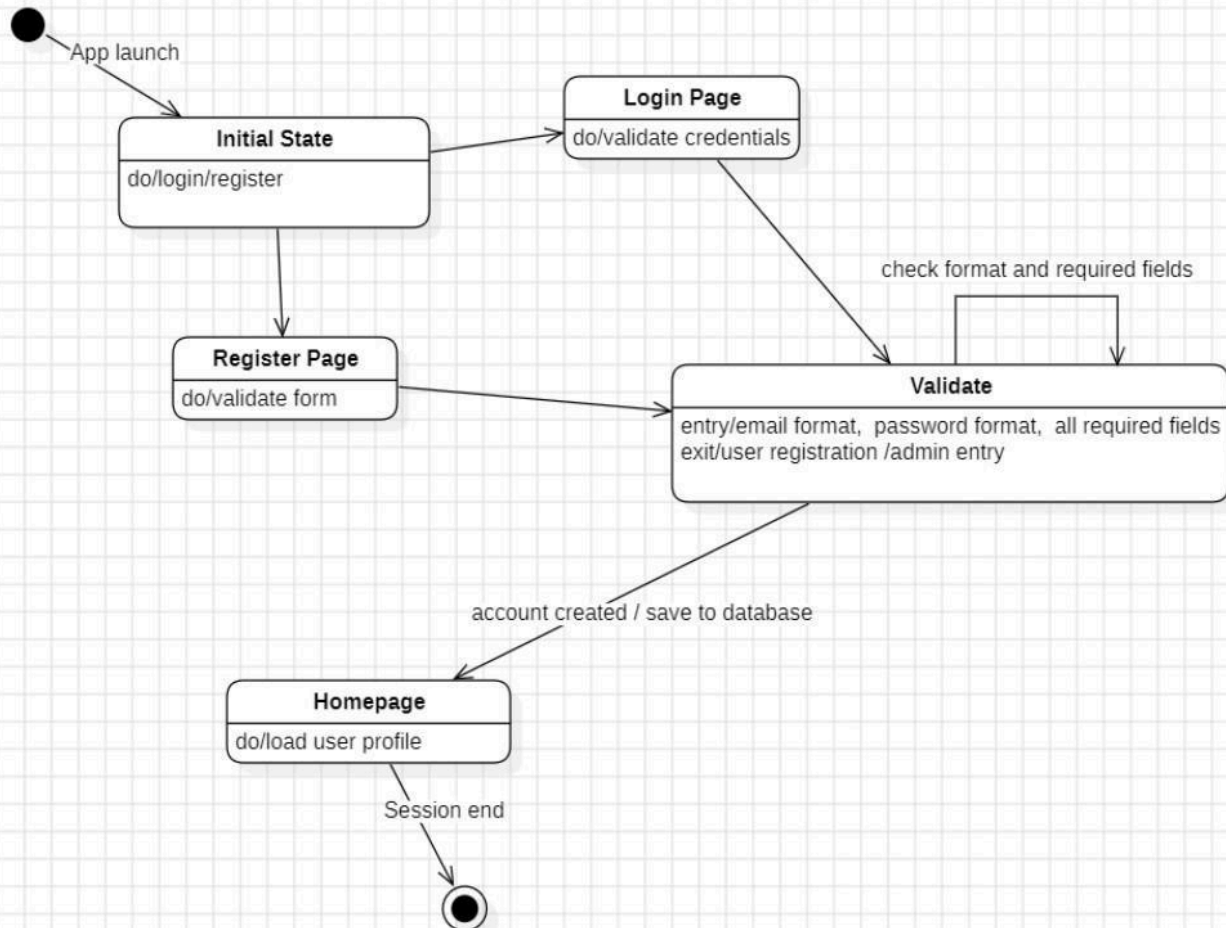
5.2 UML DIAGRAMS

5.2.1 MAIN APP MODEL

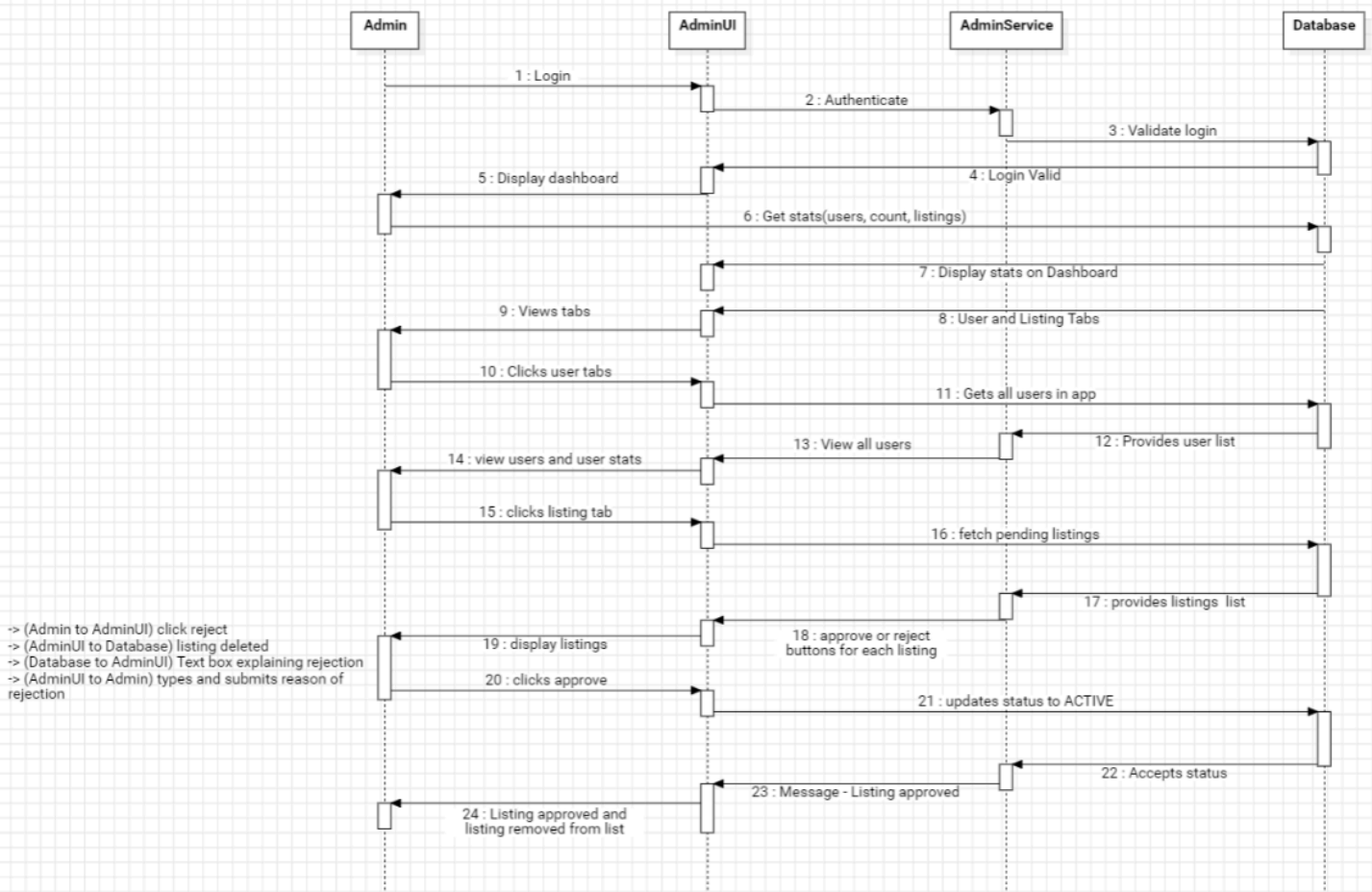


5.2.2 STD – LOGIN AND REGISTRATION

UML-STD-Login&Registration



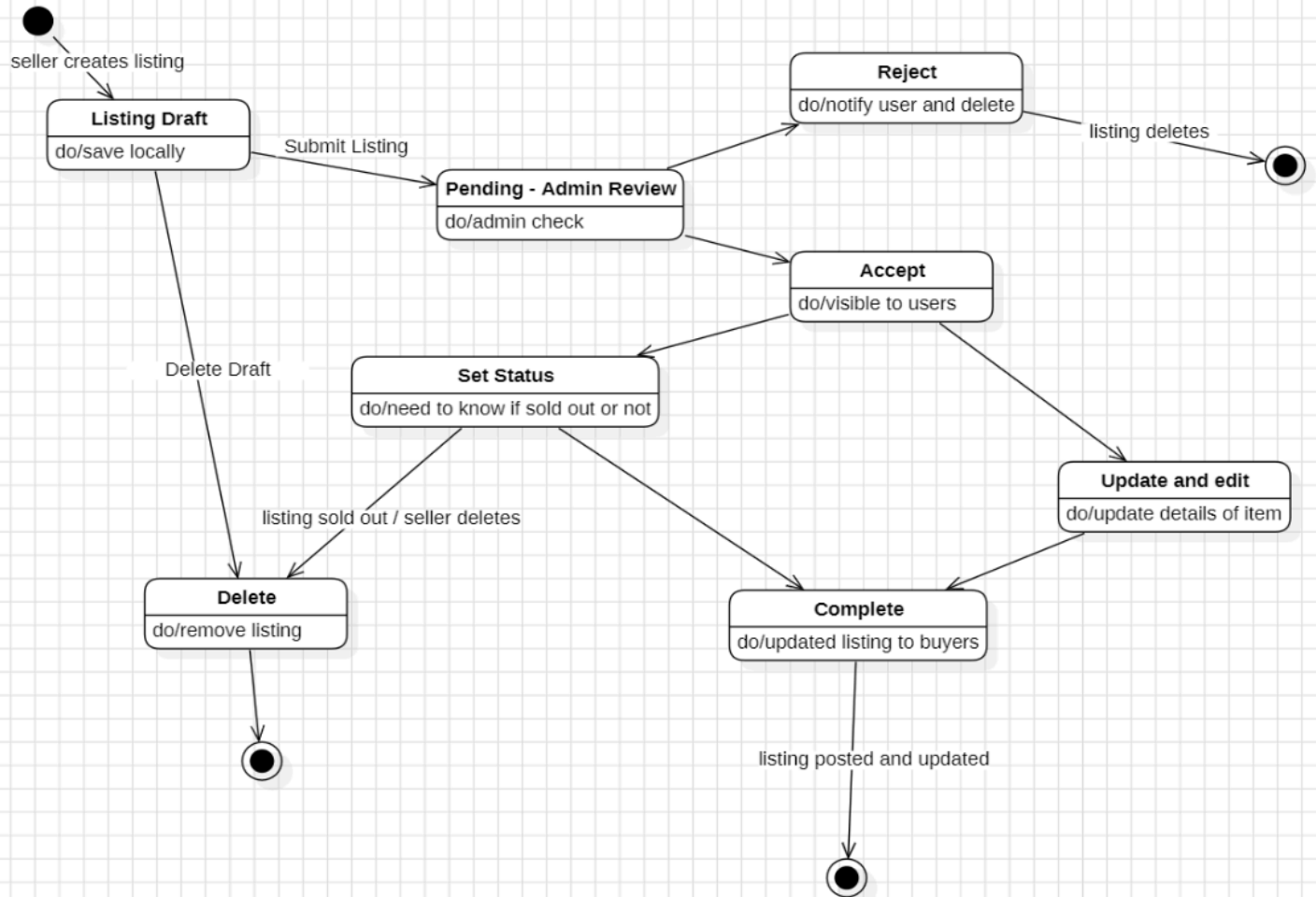
5.2.3 SEQUENCE – ADMIN AND LISTINGS



UML-Sequence-Admin&Listings

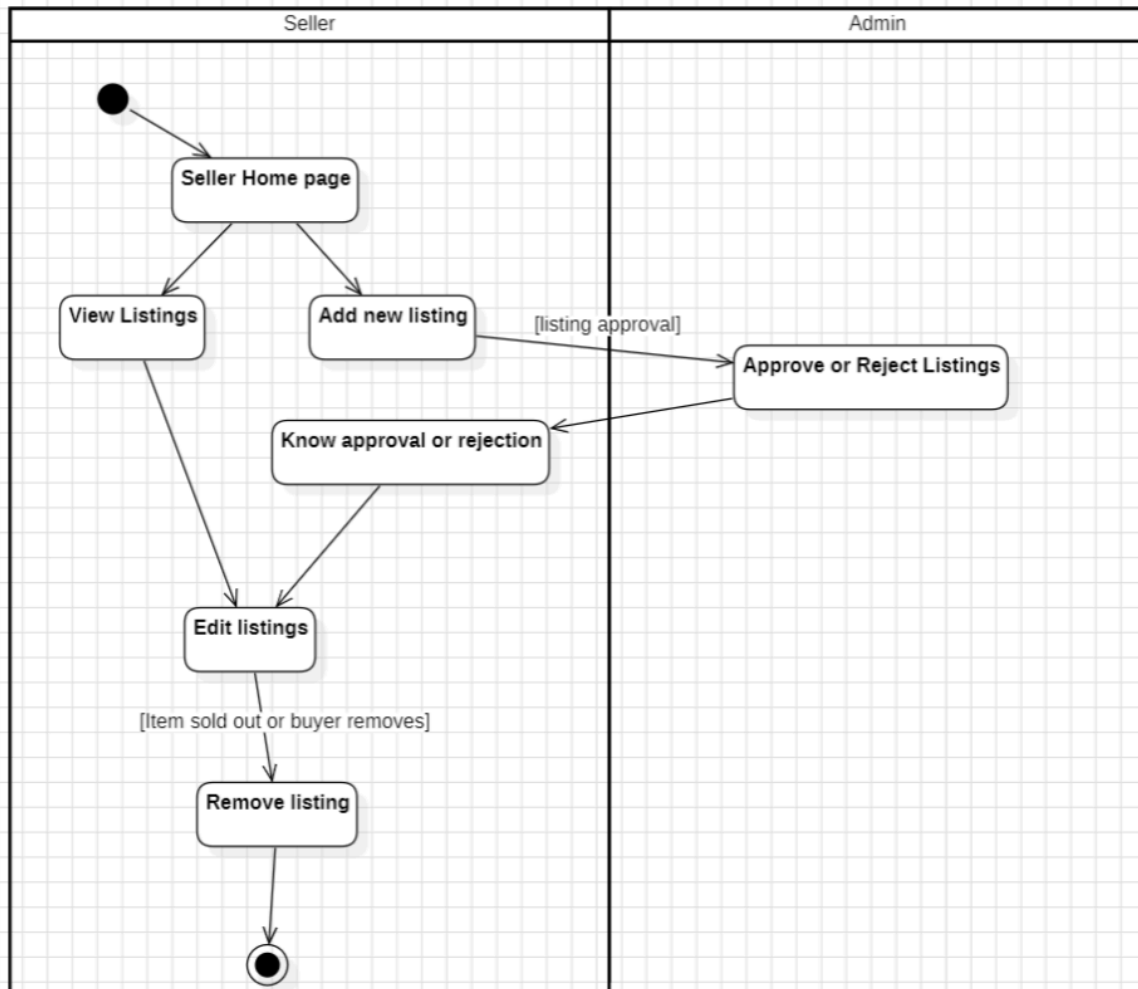
5.2.4 STD – SELLER AND LISTINGS

UML-STD-Seller&Listings



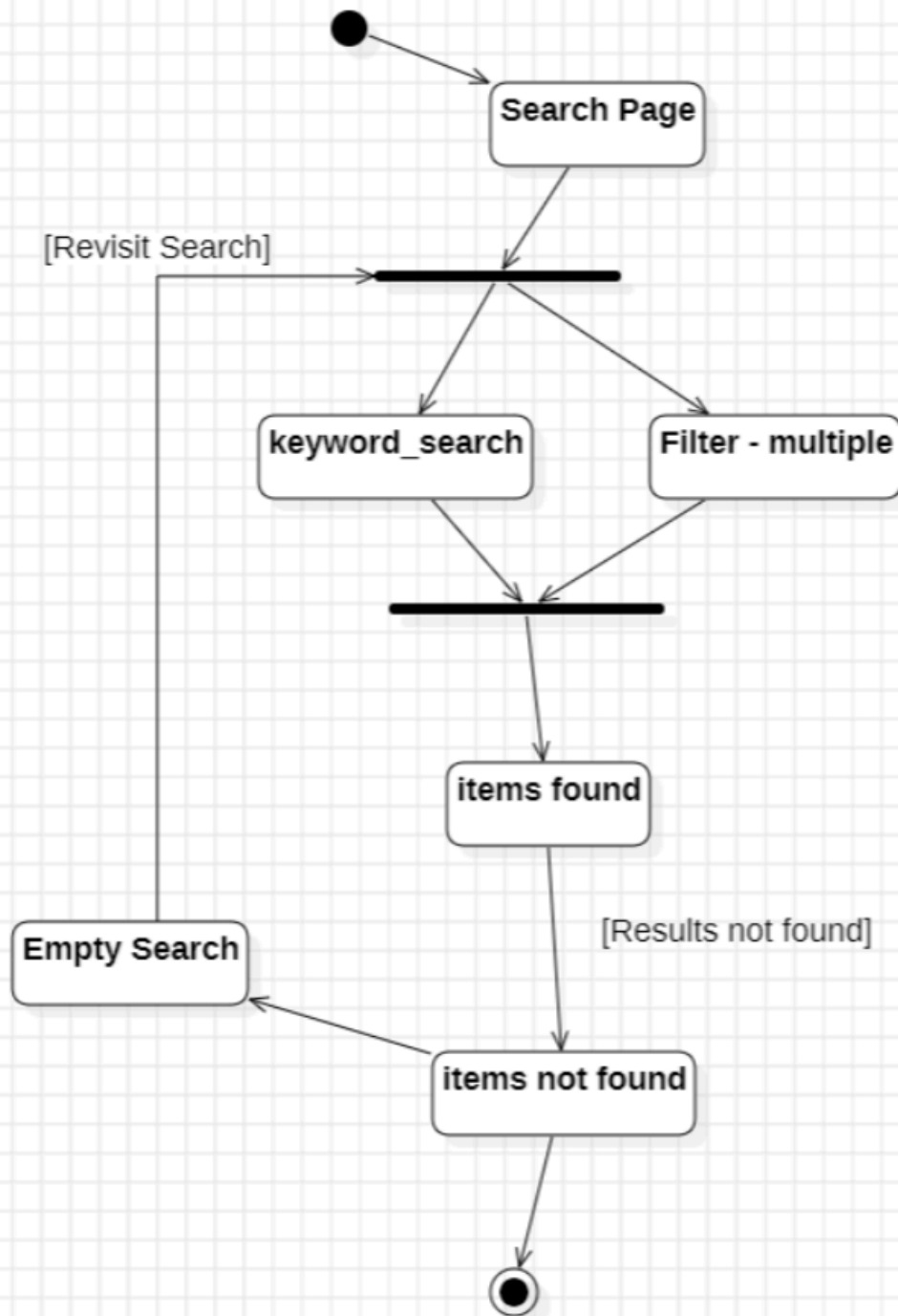
5.2.5 ACTIVITY – LISTING LIFECYCLE

UML-Activity- Listing Lifecycle

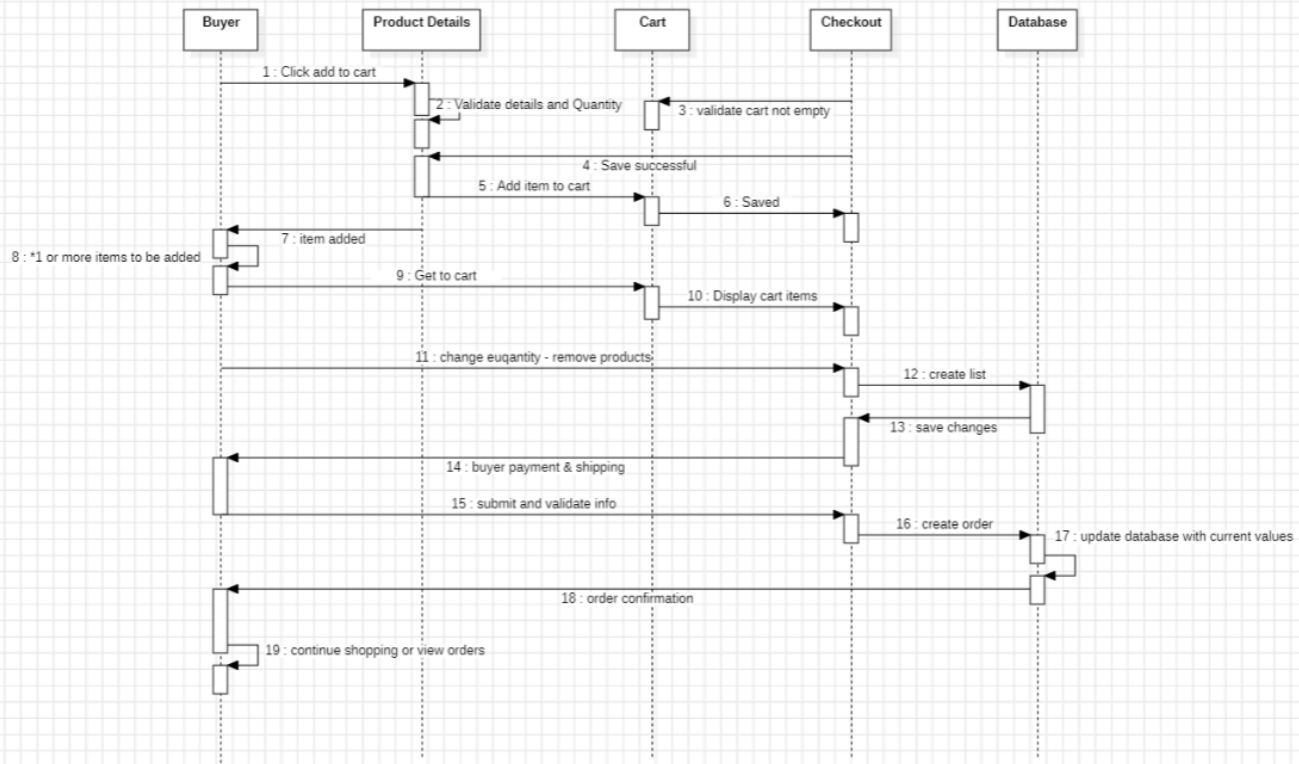


5.2.6 ACTIVITY – SEARCH AND FILTER

UML-Activity-Search&Filter



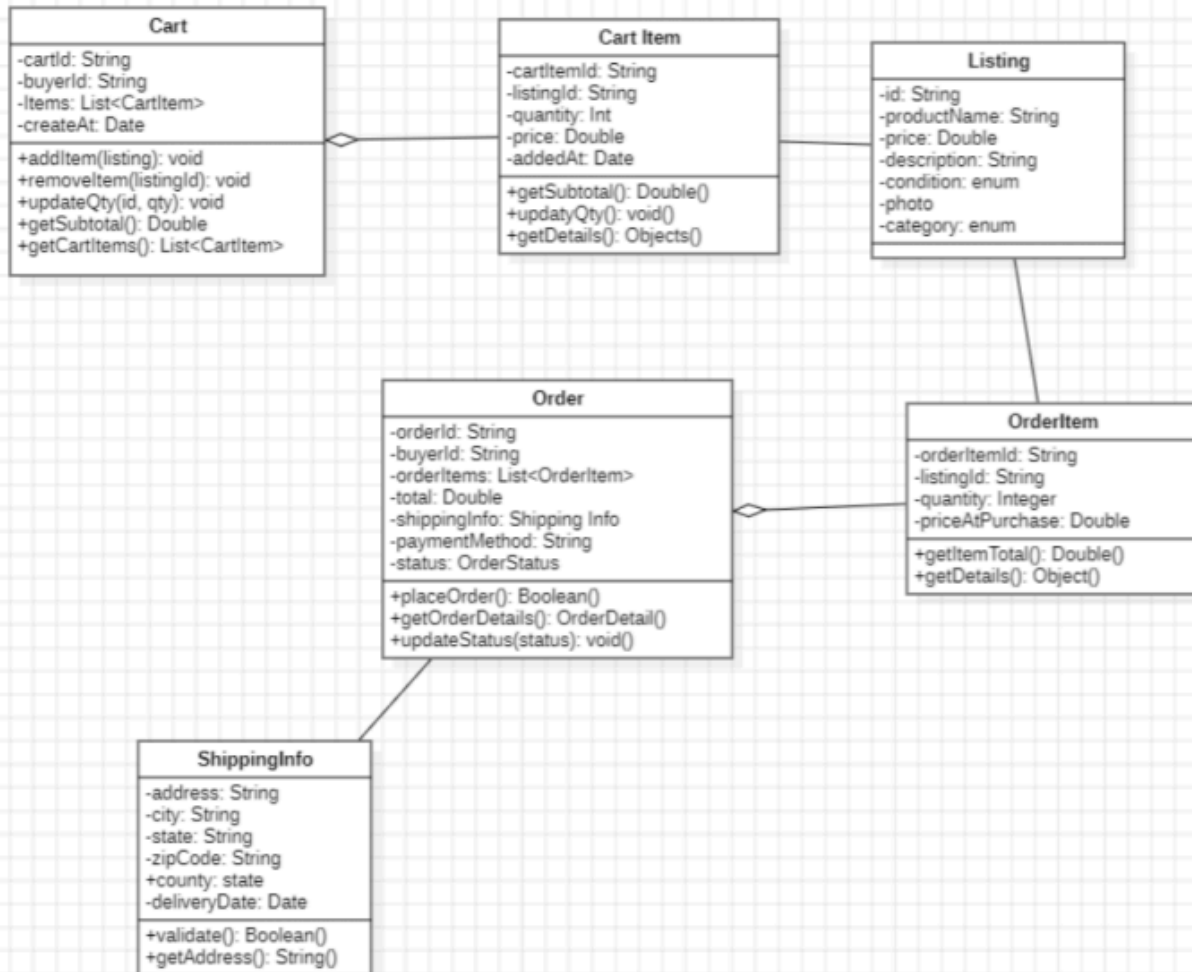
5.2.7 CLASS –CART COMPONENTS



UML-Sequence-Cart&Checkout

5.2.8 SEQUENCE – CART AND CHECKOUT

UML-Class-CartComponents



5.3 TEST PLAN

5.3.1 INTRODUCTION AND PLAN OF APPROACH

Our comprehensive testing strategy prioritizes core authentication flows (Registration and Login) as the foundational layer, ensuring user accounts can be created and authenticated securely before proceeding with any other system functionality. We then proceed to dashboard navigation and core buyer flows (Search, Filter, Browse Listings), followed by seller-specific functionality (Listing Management), admin moderation features (Admin Dashboard), and finally end-to-end buyer journeys (Shopping Cart, Checkout, Order Confirmation). This sequential approach ensures that foundational features are tested and validated before dependent features, reducing the risk of cascading failures and making it easier to identify and isolate root causes of issues.

Each module encompasses both happy-path scenarios (valid inputs and correct behavior) and sad-path scenarios (invalid inputs, missing data, error handling), ensuring robust validation of all system features under both normal and exceptional conditions. All test cases include specific expected results with exact error messages, screen transitions, database outcomes (such as entries in Firebase), and visual feedback, ensuring that test execution is objective and reproducible. This systematic approach ensures that if authentication is broken, the entire system is recognized as untestable, and we can halt testing and fix the root cause before proceeding. By testing foundational features first and maintaining clear, specific test cases throughout, we establish a reliable baseline for system quality and enable efficient debugging and resolution of any issues discovered during the testing phase.

5.3.2 TEST CASES

5.3.2.1 - REGISTRATION

Project Name: Techswap

Test Case Name: Registration

Test Case Id: CSE3310/Fall 2025/Team9/Registration

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1	User enters a valid email, username, strong password, full name, and clicks register.	A confirmation message appears. User redirected to login screen. Credentials saved in Firebase.	
TC1B	User enters valid credentials, but the password is weak, and clicks on register.	Error message: 'Password must be at least 6 characters'. Form remains visible, and the field is still required.	
TC1C	User leaves the password field empty, clicks register.	Error message: 'Password is required'. Form remains visible, focus returns to the password field.	
TC2A	User leaves the password field empty. Click 'Sign in'.	Error message: 'Password is required'. Login form remains visible, focus returns to password field.	
TC3	User with 'Seller' roles enters correct credentials. Click 'Sign in.'	Users are authenticated. Dashboard loads with 'Seller' role badge, and sellers can see many items available.	
TC4	User with 'Admin' role enters correct credentials. Click 'Sign in.'	User authenticated, Admin Dashboard loads with admin widgets and statistics.	

5.3.2.2 - LOGIN

Project Name: Techswap

Test Case Name: Login

Test Case Id: CSE3310/Fall 2025/Team9/Login

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1	User enters a valid registered username and correct passwords. Click 'Sign in.'	User authenticated successfully. Dashboard loads with role badge visible (Buyer). Session established.	
TC1B	User enters a valid username but an incorrect password. Click 'Sign in.'	Error message: Invalid username or password. User remains on the login screen. An error message appears at the bottom of the page.	
TC1C	User enters an unregistered email with any password. Click 'Sign in.'	Error message: Invalid email or password. Login screen remains visible.	
TC2A	User leaves the password field empty. Click 'Sign in.'	Error message: 'Password is required.' Login form remains visible, focus returns to password field.	
TC3	User with 'Admin' role enters correct credentials. Click 'Sign in.'	User authenticated, Admin Dashboard loads with admin widgets and statistics.	

5.3.2.3 – ELECTRONIC LISTINGS

Project Name: Techswap

Test Case Name: Electronic Listings

Test Case Id: CSE3310/Fall 2025/Team9/Electronic_Listings

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1	User (Buyer) logs in and navigates to Browser Listings; Database contains approved listings.	Listings page loads. All listings display images, names, brands/models, prices, and conditions. Responsive and scrollable.	
TC1B	User clicks on an individual listing to view details.	Listing detail page opens with full images, description, price, condition, and 'Add to cart' button.	
TC1C	User scrolls through the list of listings.	Smooth scrolling behavior. All cards are rendered correctly. Images load progressively.	
TC2	User accesses listings when no listings exist in the database.	The listings page displays. 'No listings available.' With the option to return to the home page.	

5.3.2.4 SEARCH AND FILTER

Project Name: Techswap

Test Case Name: Search & Filter

Test Case Id: CSE3310/Fall 2025/Team9/Search_Filter

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1	User applies a single filter: Category = 'Phones'.	The listing page displays only phones. Count updates. Other filters are visible for refinement.	
TC1B	User applies filters that result in no matches.	Page displays: No listings match your filters.	

5.3.2.5 – SELLER LISTING MANAGEMENT

Project Name: Techswap

Test Case Name: Seller Listing Management

Test Case Id: CSE3310/Fall 2025/Team9/Seller_Listing_Management

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1	Seller navigates to Create Listing. Fills all fields (name, price, condition, description, images). Click Submit	Listing saved in Firebase with status = 'Pending Admin Review.' Appears in 'My Listings' with 'Pending Approval' badge (not visible to buyers).	
TC1B	Seller creates listings but leaves the Price field empty. Click Submit	Error message: 'Price is required.' Form remains open, all data intact. Price field highlighted.	
TC1C	Seller creates listings but does not upload images. Click Submit.	Error message: At least one product image is required. The form remains open.	
TC1D	Seller cancels the Create Listing form. Click the back button before submission.	The form closes without saving. User returned to Seller Home. No partial data saved.	

5.3.2.6 – ADMIN DASHBOARD

Project Name: Techswap

Test Case Name: Admin Dashboard

Test Case Id: CSE3310/Fall 2025/Team9/Admin_Dasboard

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1A	Admin logs in. Admin Dashboard loads.	Dashboard displays: total user count, total listing count, pending listings count, and disabled accounts count. Stats current	
TC1B	Admin navigates to the Users tab in the dashboard.	List of all users displays with columns: username, email, role (Buyer/Seller/Admin), status (Active/Disabled).	
TC2A	Admin views Pending Listings. See listings violating guidelines. Click Reject.	Modal appears asking for rejection reasons. Admin enters the reason click Confirm. Listing status 'Rejected', removed from the marketplace. – login error	
TC2B	Admin approves a seller's pending listings.	Listing status 'Active'. Visible to all buyers in the marketplace.	
TC3A	Admin views the user account as violating policies. Click Disable Account. Confirms.	User status 'Disabled'. The user cannot log in. User's listings hidden from buyers.	
TC3B	Admin clicks Enable Account on the disabled account.	Account status 'Active'. The user can log in. Listings visible to buyers (if approved).	
TC4	Admin deletes listing violating policies.	Listing removed from the marketplace. Status 'Deleted by Admin'.	

5.3.2.7 SHOPPING CART

Project Name: Techswap

Test Case Name: Shopping Cart

Test Case Id: CSE3310/Fall 2025/Team9/Shopping_Cart

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1A	Buyer views the listing and clicks Add to Cart.	Item added to the cart badge. cart.'	
TC1B	Buyer adds multiple items (iPhone, laptop, tablet) to cart.	Each item was added successfully.	
TC2A	Buyer clicks the cart icon to view the cart contents.	Cart page opens showing all items with prices, total calculated correctly.	
TC2B	Buyer removes item from cart.	Item deleted. Total price recalculates. Confirmation message shown.	
TC2C	Buyer navigates back to listings from cart.	User returns to listings successfully. Cart preserved (items remain).	
TC3	Buyer clicks Proceed to Checkout from the cart page.	User taken to checkout/payment screen. Cart contents passed to the next step.	

5.3.2.8 CHECKOUT/PAYMENT

Project Name: Techswap

Test Case Name: Checkout/Payment

Test Case Id: CSE3310/Fall 2025/Team9/Checkout_Payment

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1A	Buyer arrives at checkout with items in cart. Reviews order summary.	Order summary displays: all items listed, individual prices correct, total calculated accurately.	
TC1B	Buyer enters valid mock payment (Card#, Exp, CVV, and Name). Click Place order.	Payment processed (mock). Order created in Firebase with status 'Confirmed'. Order ID displayed. Cart cleared. User redirected to confirmation.	
TC2A	Buyer enters invalid card number. Attempt to place order.	Error message: 'Invalid card number. Please check your card details.' Order not created. Cart intact, Form open.	
TC2B	Buyer enters expired card. Attempts to place order.	Error message: 'Card has expired. Please use a valid card.' Order not created. Form open.	
TC2C	Buyer leaves CVV field empty. Clicks Place Order.	Error message: 'CVV is required' Form open, other fields preserved.	
TC2D	Buyer leaves Cardholder Name field empty. Clicks Place Order.	Error message: 'Cardholder name is required.' Form open.	
TC3A	Buyer reviews billing and clicks 'Confirm.'	Proceeds to final order placement. Order submitted.	
TC4A	Buyer clicks 'Cancel Checkout' before placing order.	User returned to shopping cart. Cart content preserved. No order created.	

5.3.2.9 ORDER CONFIRMATION

Project Name: Techswap

Test Case Name: Order Confirmation

Test Case Id: CSE3310/Fall 2025/Team9/Order Confirmation

Test Case No.	Test Case Description	Expected Results	Outcome Pass, Fail, Other (comments)
TC1A	The user completes the checkout successfully. Confirmation of screen loads.	Display Order Confirmation Number purchased items, and total amount paid.	
TC1B	The Confirmation page shows the order number as unique values across all orders.	Order numbers that are unique and non-repeat. No two orders share the same number.	
TC2	Users view Total Amount Paid on a confirmation screen.	Amount accurate (sum of all items). Matches checkout amount.	
TC3	User clicks the Return to Home button on the confirmation screen.	The user was taken to the homepage. Cart empty. Users can browse listings again.	

5.4 USER MODEL

5.4.1 SPLASH SCREEN



5.4.2 LOGIN SCREEN

The image shows a mobile application login screen for 'TechSwap'. At the top, there is a logo consisting of a blue smartphone and a green laptop with two curved arrows (one blue, one green) indicating a swap or exchange. Below the logo, the text 'TechSwap' is displayed in a bold, sans-serif font, with 'Tech' in blue and 'Swap' in green. The login form includes two input fields: 'Username' and 'Password', each with a placeholder text 'Enter Username' and 'Enter Password' respectively. Below the password field is a blue 'LOGIN' button. Underneath the button are three links: 'Register', 'Forgot Username?', and 'Forgot Password?'. At the bottom of the screen, there is a copyright notice '© 2026 TechSwapAD'. The entire screen is framed by a black border, and a small black circle at the top center represents the camera notch.


TechSwap

Username
Enter Username

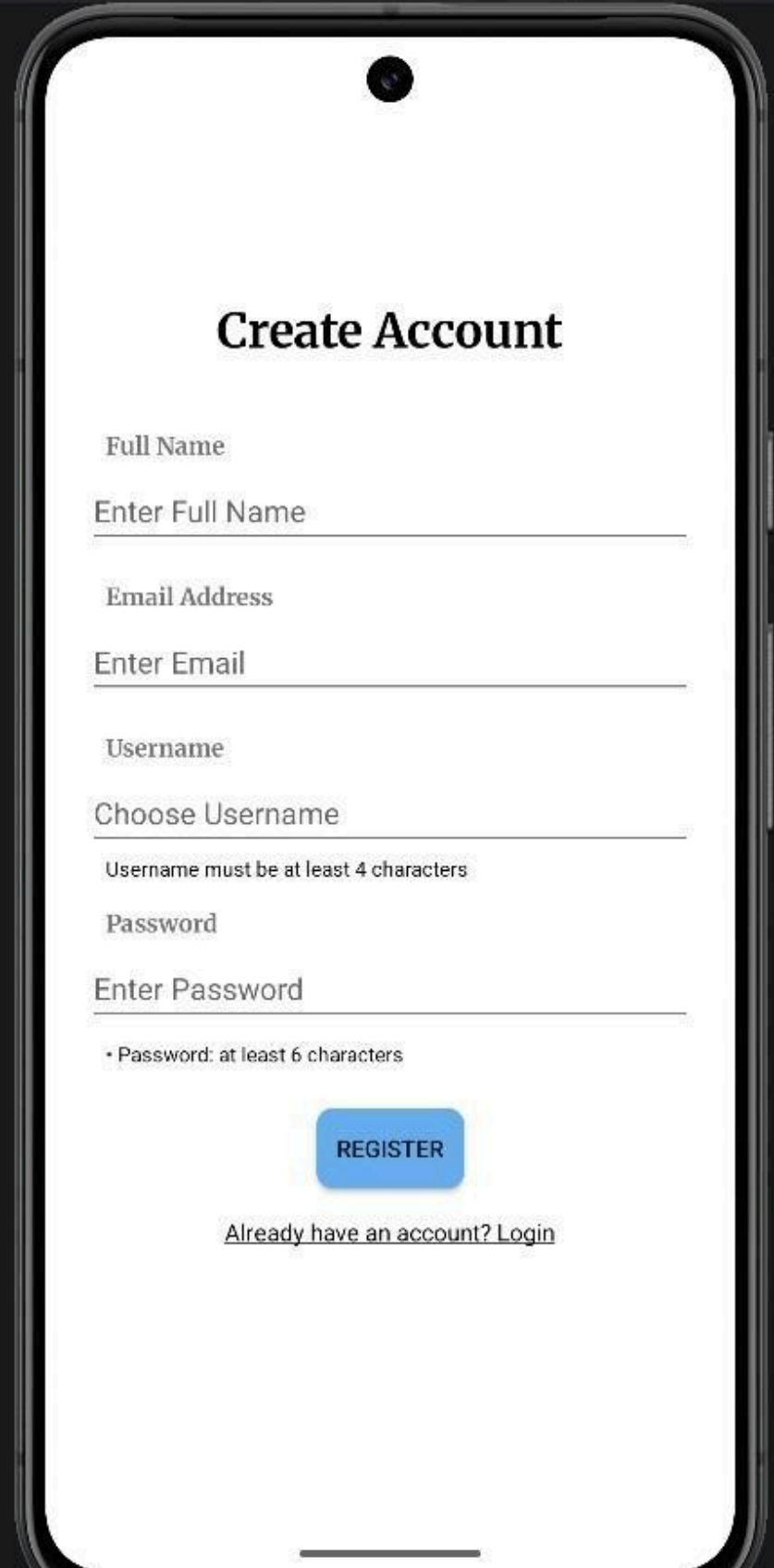
Password
Enter Password

LOGIN

[Register](#)
[Forgot Username?](#)
[Forgot Password?](#)

© 2026 TechSwapAD

5.4.3 REGISTER SCREEN

A mobile application registration screen with a white background and rounded corners. At the top center is a black circular camera notch. The title "Create Account" is centered in a large, bold, black font. Below the title are four input fields, each with a label above it and a placeholder text inside the field. The first field is labeled "Full Name" with placeholder "Enter Full Name". The second is labeled "Email Address" with placeholder "Enter Email". The third is labeled "Username" with placeholder "Choose Username". The fourth is labeled "Password" with placeholder "Enter Password". Below the "Username" field is a small error message: "Username must be at least 4 characters". Below the "Password" field is a small error message: "• Password: at least 6 characters". At the bottom center is a blue rounded rectangular button with the text "REGISTER" in white. Below the button is a link: "Already have an account? Login".

Create Account

Full Name
Enter Full Name

Email Address
Enter Email

Username
Choose Username

Username must be at least 4 characters

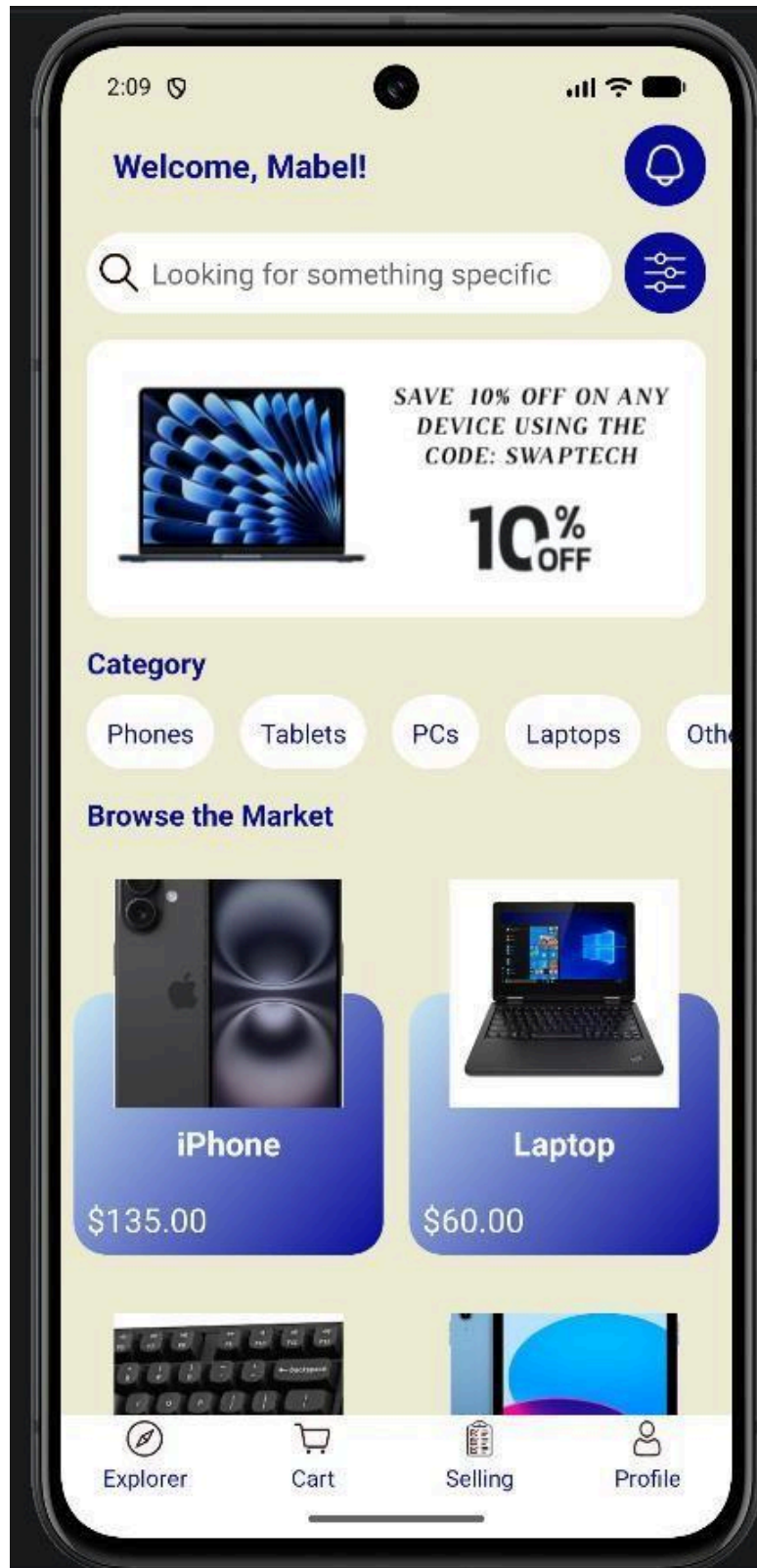
Password
Enter Password

• Password: at least 6 characters

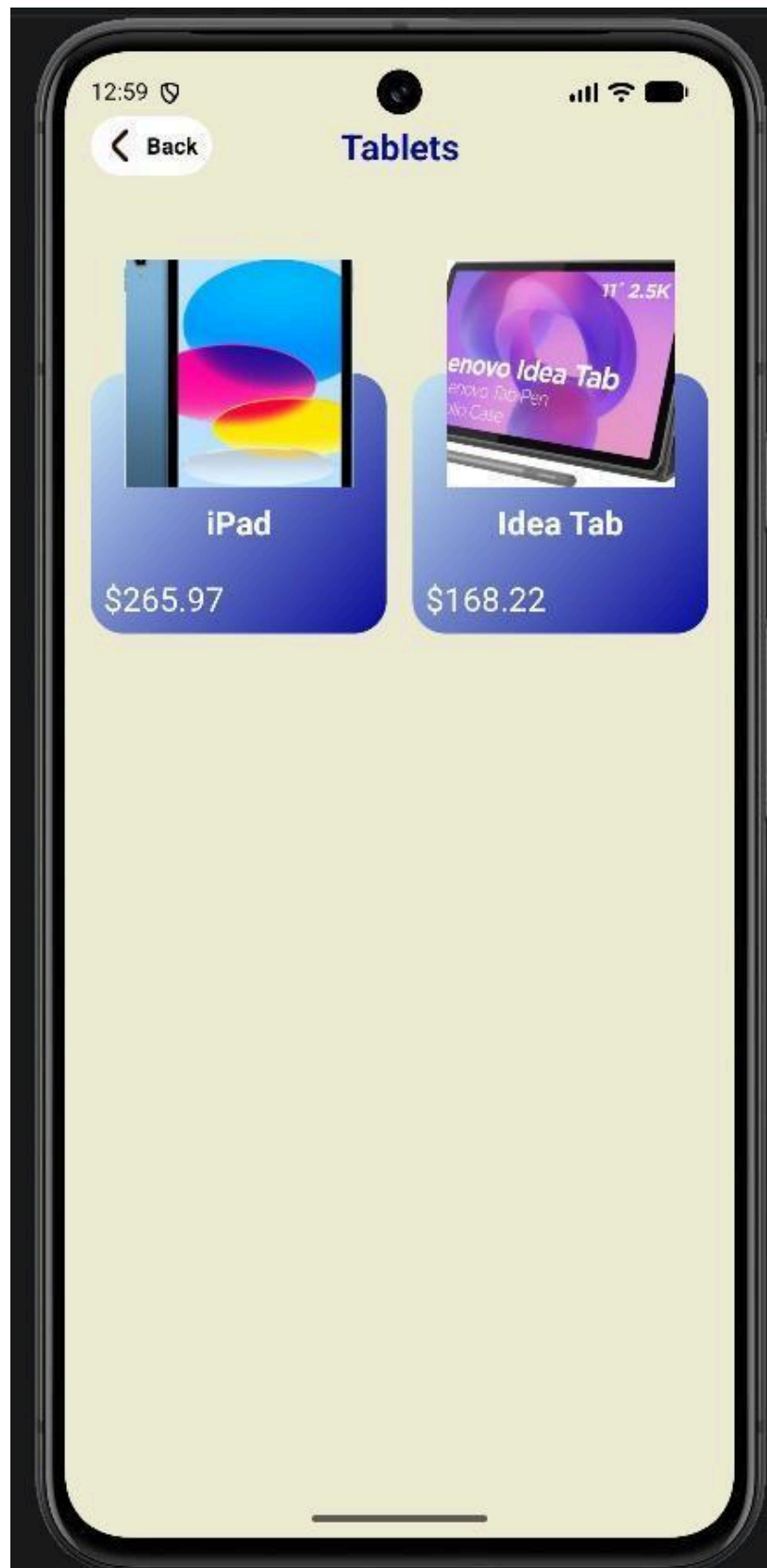
REGISTER

[Already have an account? Login](#)

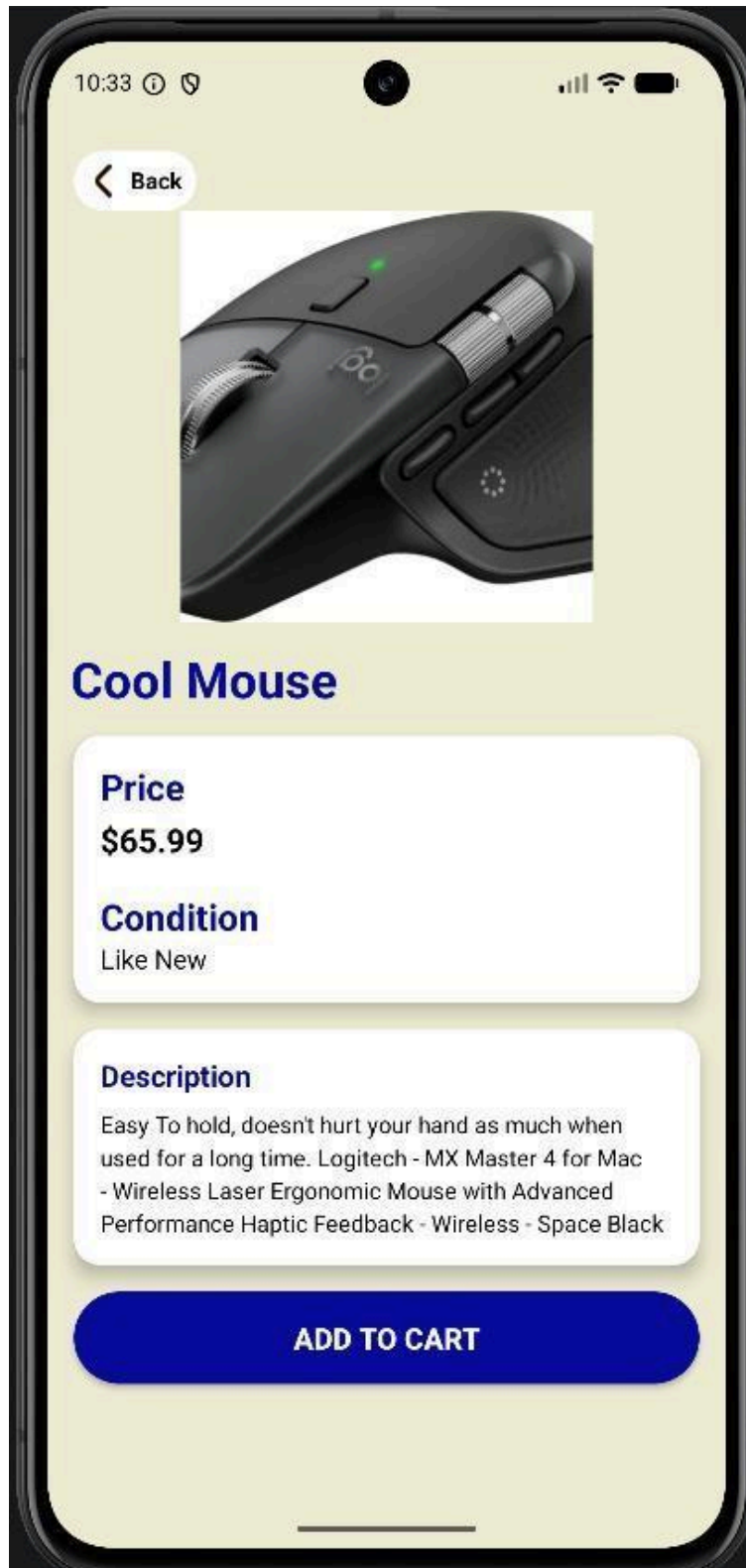
5.4.4 HOME PAGE SCREEN



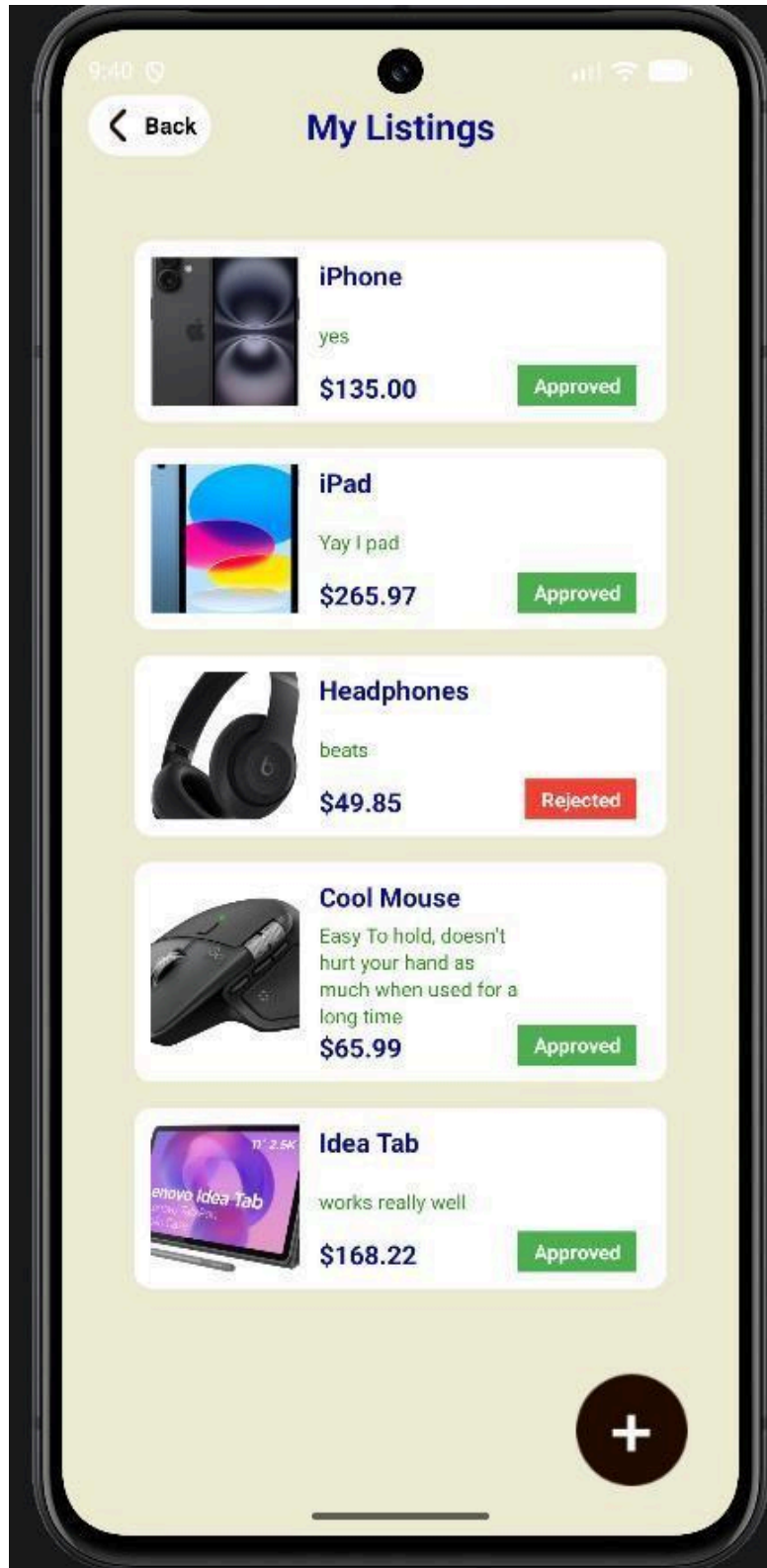
5.4.5 FILTER BY CATEGORY SCREEN



5.4.6 ITEM DETAILS SCREEN



5.4.7 SELLER DASHBOARD SCREEN



5.4.8 CREATE A LISTING SCREEN

10:35

Create Listing

Item Title

Description

Category

Phones

Condition

☐ New ☐ Like New ☐ Used

Price

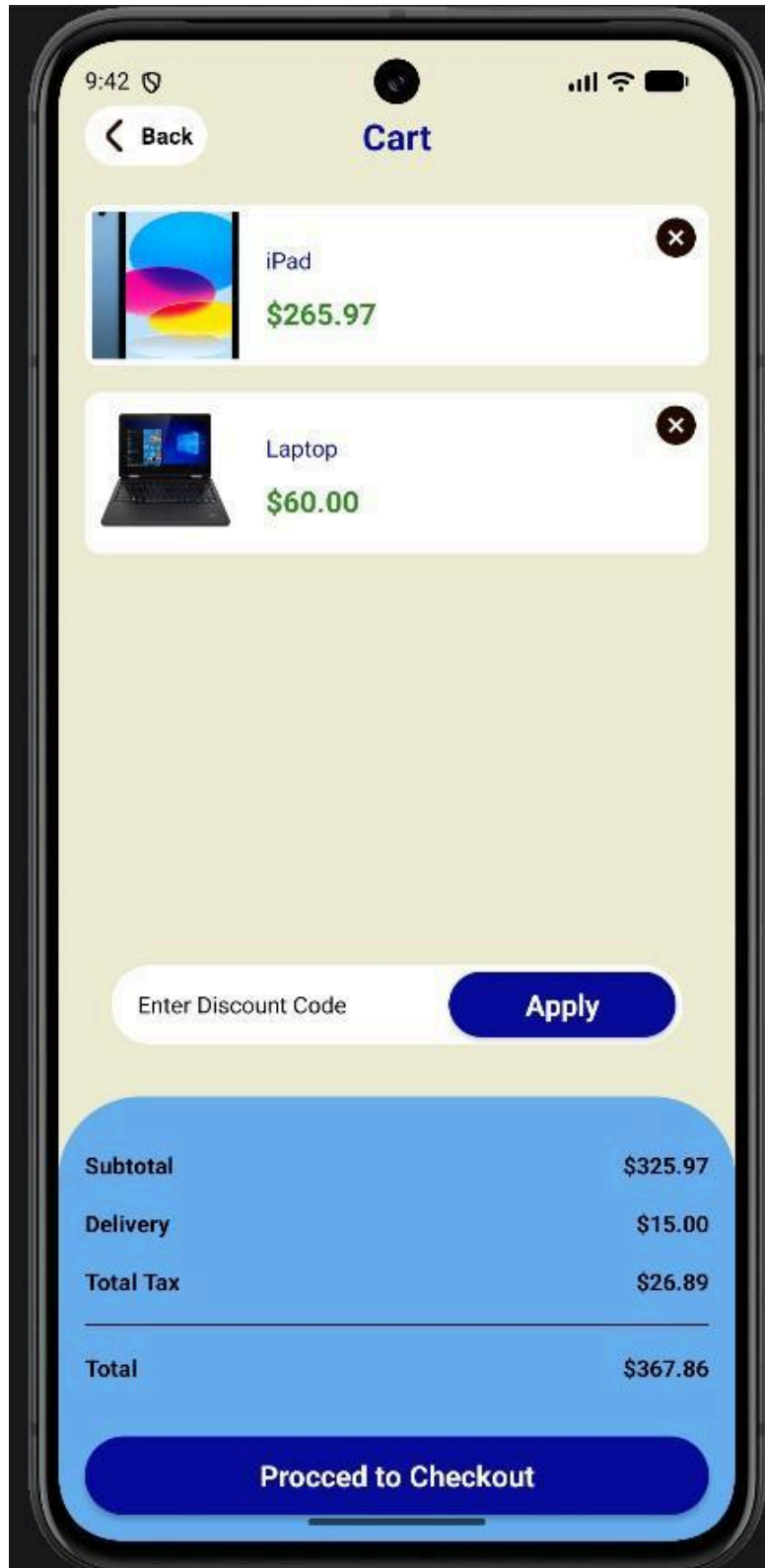
UPLOAD IMAGE

CANCEL SUBMIT

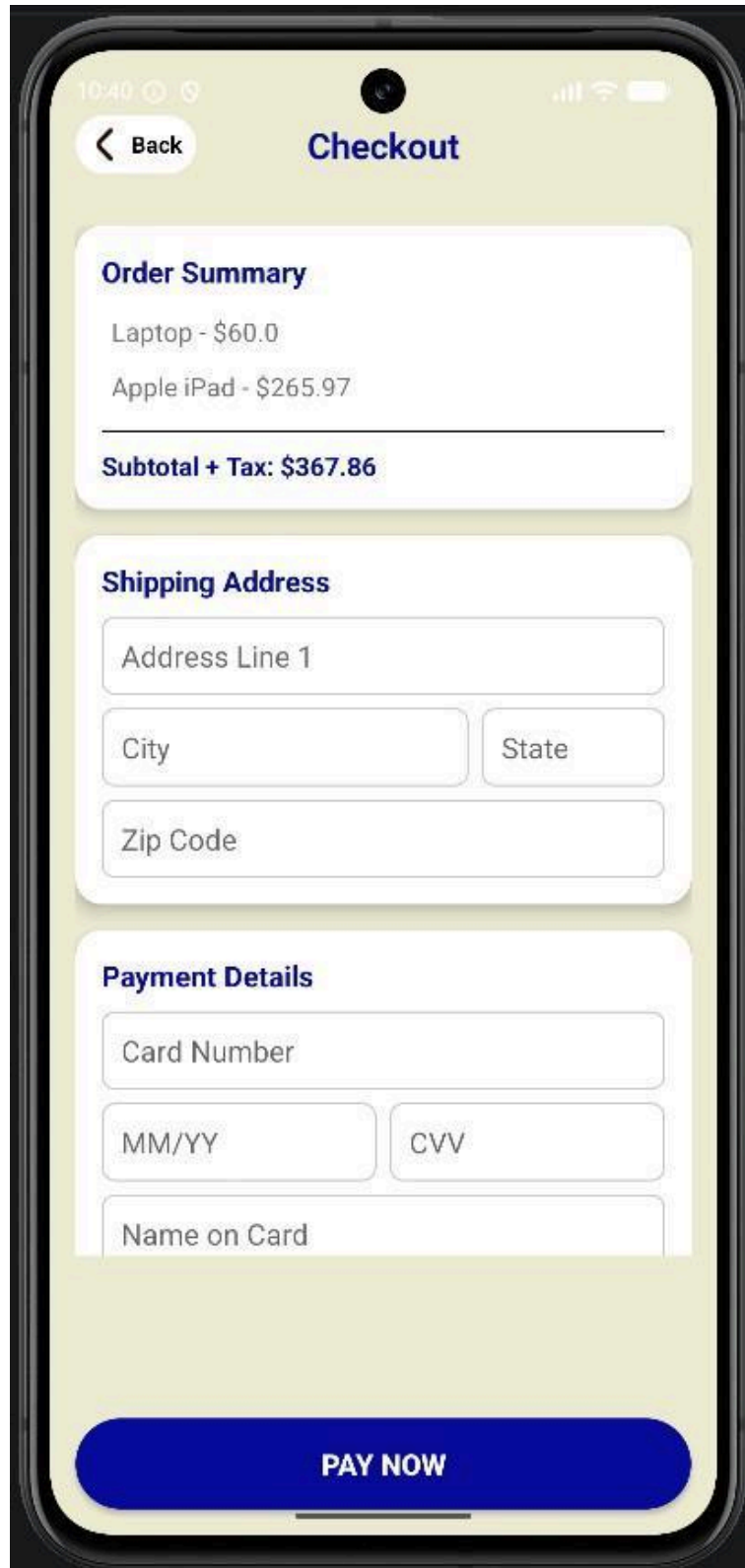
5.4.9 PROFILE PAGE SCREEN



5.4.10 CART SCREEN



5.4.11 CHECKOUT PAGE SCREEN

A mobile application checkout screen with a light yellow background. At the top, there's a status bar with the time 10:40 and various icons. Below it is a navigation bar with a 'Back' button and the title 'Checkout'. The screen is divided into three main sections: 'Order Summary', 'Shipping Address', and 'Payment Details'. The 'Order Summary' section lists 'Laptop - \$60.0' and 'Apple iPad - \$265.97', with a subtotal and tax of \$367.86. The 'Shipping Address' section has input fields for 'Address Line 1', 'City', 'State', and 'Zip Code'. The 'Payment Details' section has input fields for 'Card Number', 'MM/YY', 'CVV', and 'Name on Card'. A large blue 'PAY NOW' button is at the bottom.

10:40

[Back](#) Checkout

Order Summary

Laptop - \$60.0

Apple iPad - \$265.97

Subtotal + Tax: \$367.86

Shipping Address

Address Line 1

City State

Zip Code

Payment Details

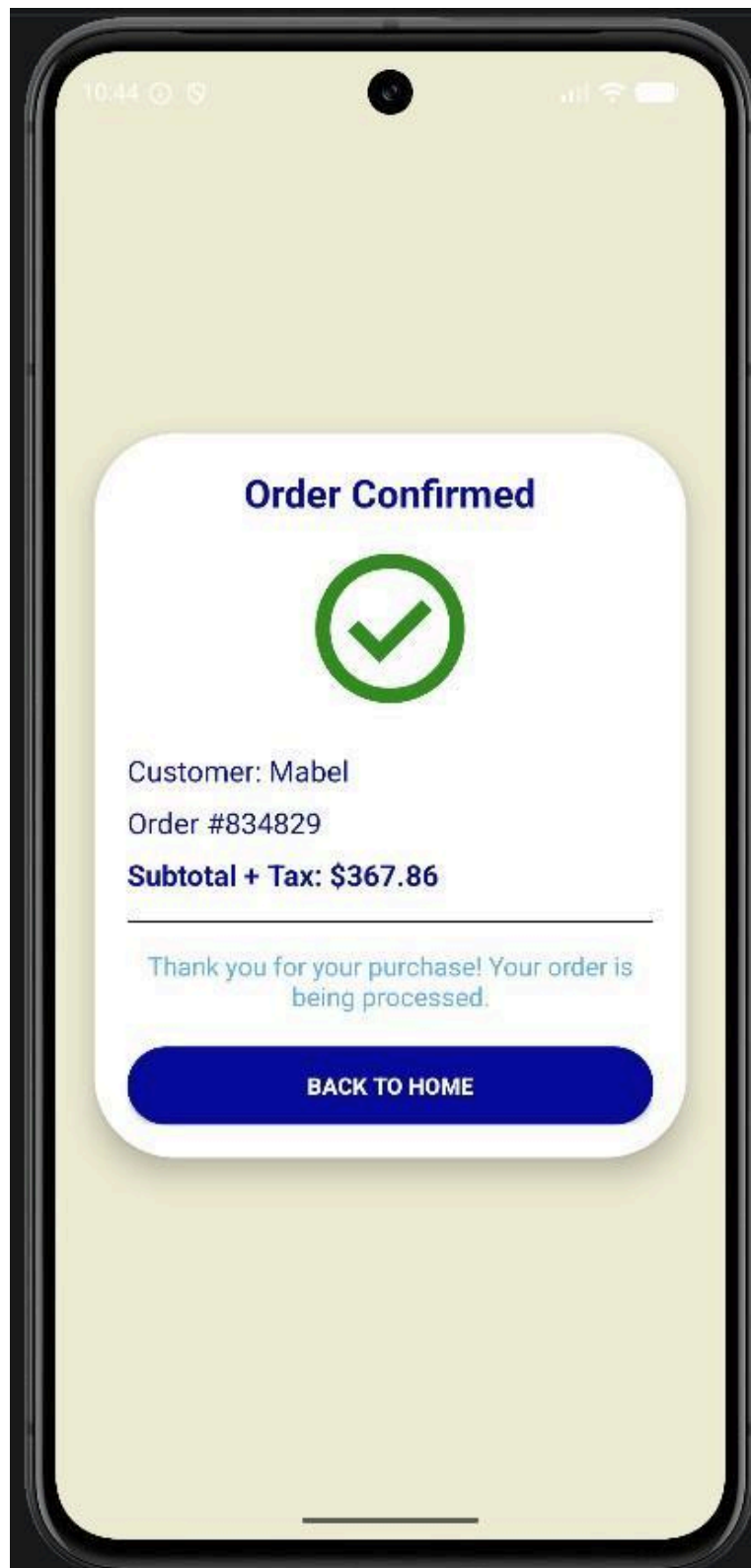
Card Number

MM/YY CVV

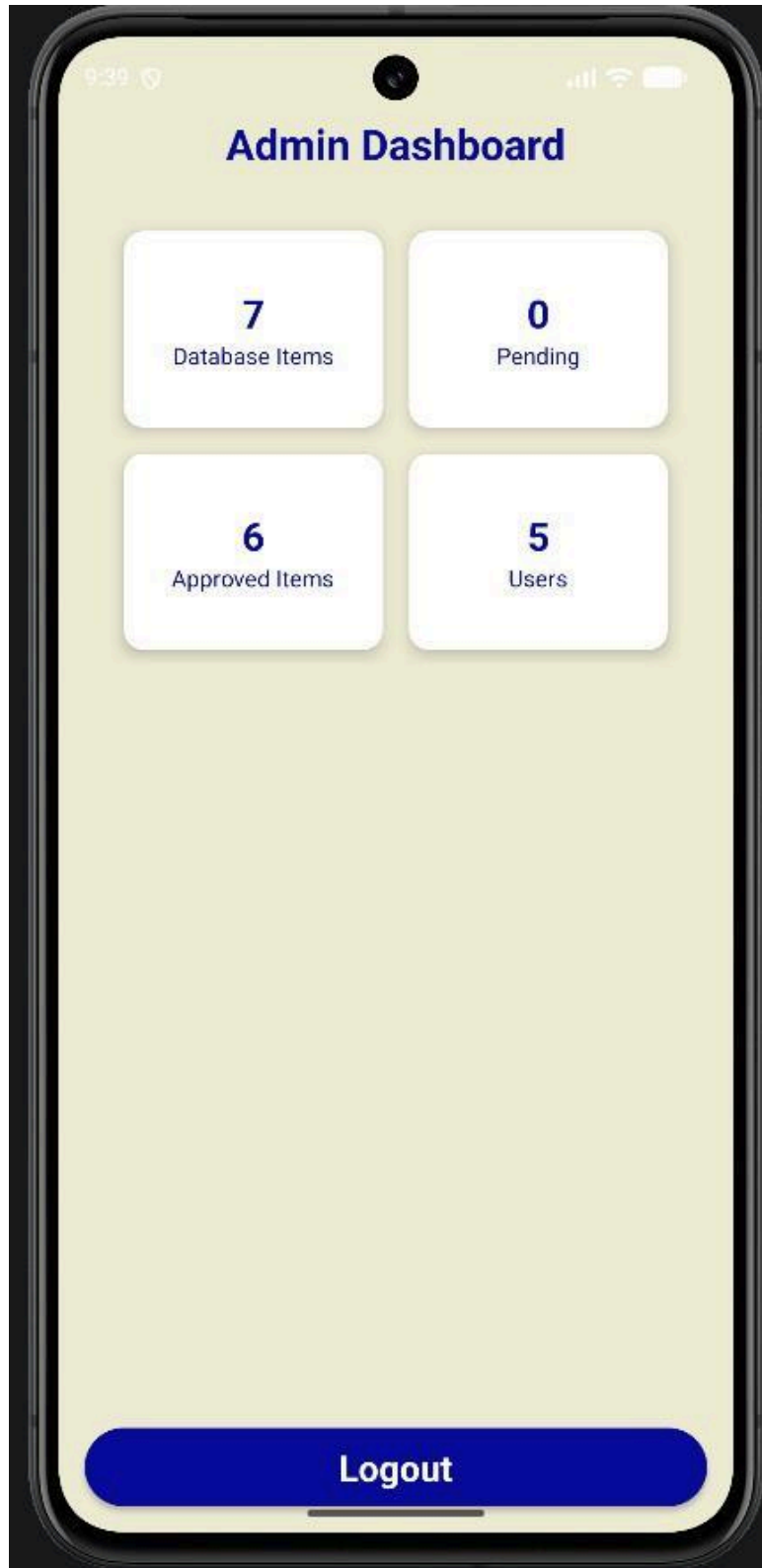
Name on Card

PAY NOW

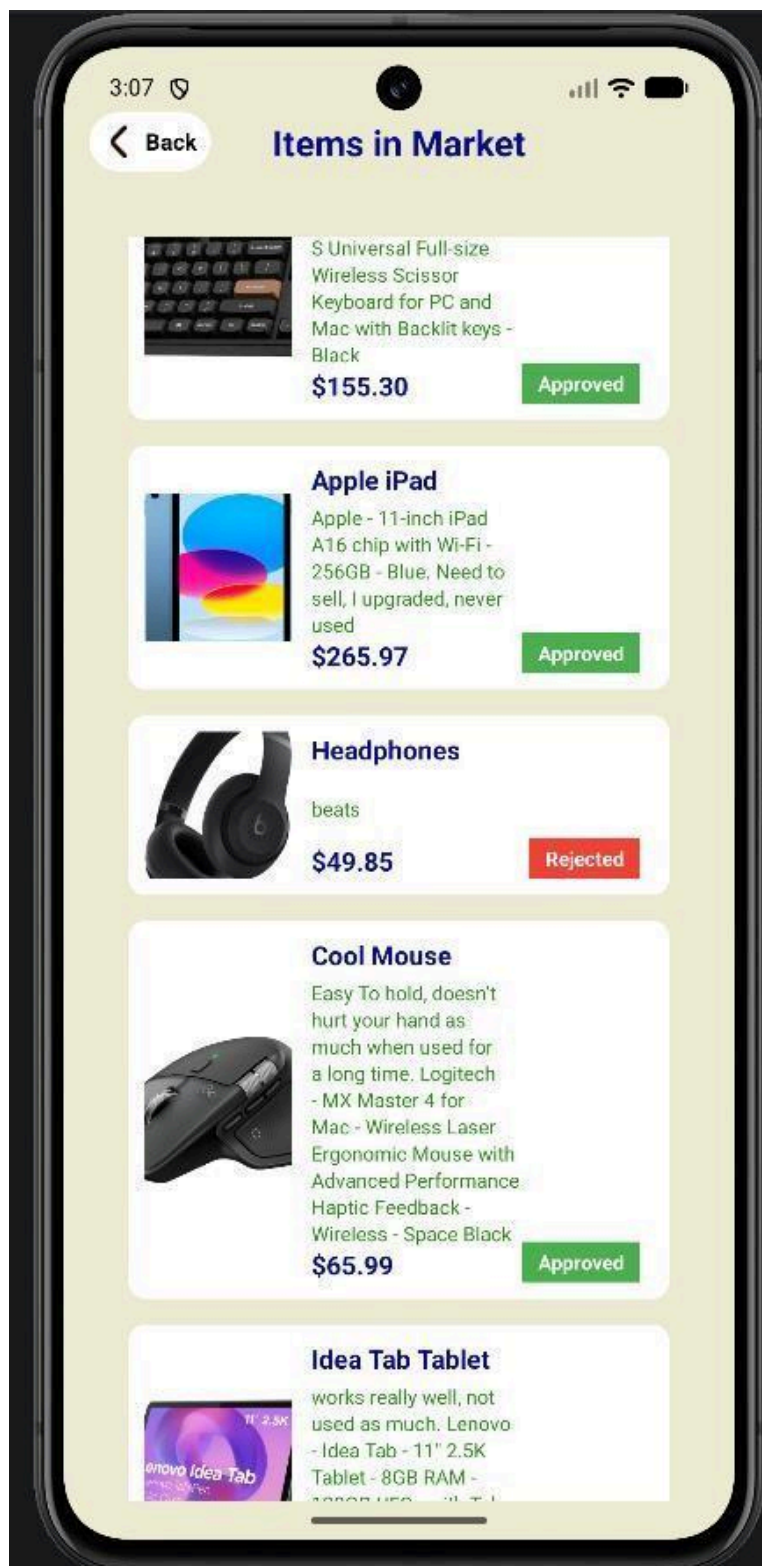
5.4.12 ORDER CONFIRMATION SCREEN



5.4.13 ADMIN DASHBOARD SCREEN



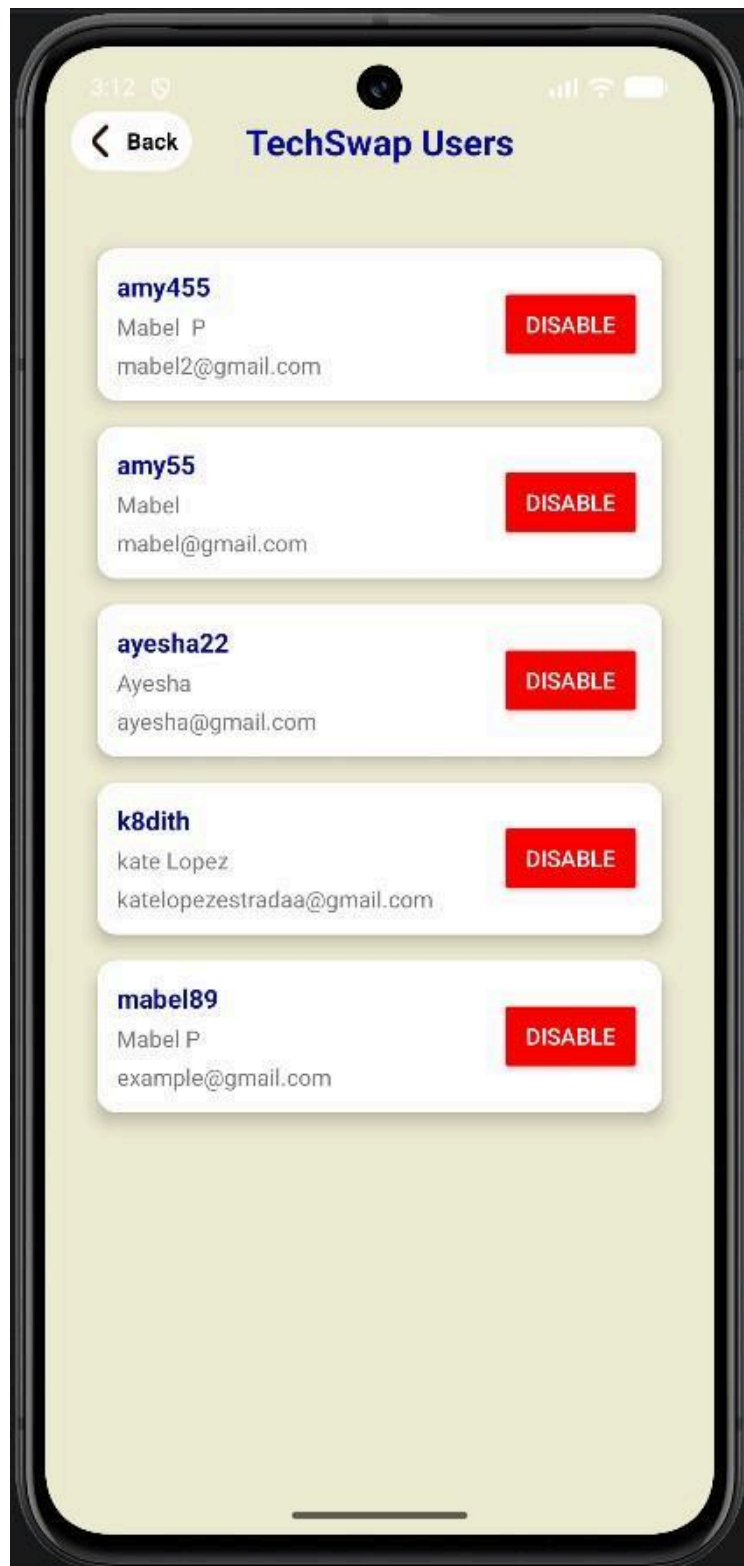
5.4.14 VIEW ALL LISTINGS SCREEN



5.4.15 VIEW PENDING ITEMS SCREEN



5.4.16 VIEW ALL USERS SCREEN



5.5 MAIN SELECTED SOURCE CODE

5.5.1 LOGIN ACTIVITY CODE

```
package com.example.loginpage.Activity

import android.content.Intent import
    android.os.Bundle import
    android.widget.Button import
    android.widget.EditText import
    android.widget.TextView import
    android.widget.Toast import
    androidx.appcompat.app.AlertDialog import
    androidx.appcompat.app.AppCompatActivity
import com.example.loginpage.Helper.ManagmentCart
import com.example.loginpage.R
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.database.FirebaseDatabase
import com.google.firebase.firestore.FirebaseFirestore
class LoginActivity : AppCompatActivity() {
    private lateinit var auth: FirebaseAuth
    private lateinit var db: FirebaseFirestore

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_login)

        auth =
            FirebaseAuth.getInstance()
        db = FirebaseFirestore.getInstance()

        val usernameInput = findViewById<EditText>(R.id.LoginNametxt)
        val passwordInput = findViewById<EditText>(R.id.editTextTextPassword)
        val loginButton = findViewById<Button>(R.id.Loginbutton)
        val registerButton = findViewById<TextView>(R.id.registerButton)
        val forgotPassword = findViewById<TextView>(R.id.forgotPassword)
        val forgotUsername = findViewById<TextView>(R.id.forgotUsername)

        val adminAccess = findViewById<TextView>(R.id.adminAccess)

        //----- checking valid login information
        -----// //LOGIN
        loginButton.setOnClickListener {
            val input = usernameInput.text.toString().trim()
            val password = passwordInput.text.toString().trim()

            if (input.isEmpty() || password.isEmpty()) {
                Toast.makeText(this, "Please enter username/email and password", Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

```
        return@setOnClickListener  
    }  
  
    if (input.contains("@")) {  
        loginWithEmail(input, password)  
        return@setOnClickListener    }
```

```

db.collection("users").document(input).get()
.addOnSuccessListener { document ->
if (!document.exists()) {
    Toast.makeText(this, "Username not found",
Toast.LENGTH_SHORT).show()
        return@addOnSuccessListener
    }

    val email = document.getString("email")

    if (email.isNullOrEmpty()) {
        Toast.makeText(this, "Email/Username not found for
user", Toast.LENGTH_SHORT).show()
        return@addOnSuccessListener
    }
    loginWithEmail(email, password)
}
.addOnFailureListener { e ->
    Toast.makeText(this, "Lookup failed: ${e.message}",
Toast.LENGTH_SHORT).show()
}
}

//REGISTER
registerButton.setOnClickListener {
    val intent = Intent(this, RegisterActivity::class.java)
startActivity(intent)
}

//FORGET PASSWORD
forgotPassword.setOnClickListener {
showEmailDialog("Reset Password") { email ->
auth.sendPasswordResetEmail(email)
        .addOnSuccessListener {

```

```

        Toast.makeText(this, "Reset link sent to your email",
Toast.LENGTH_LONG)
            .show()
    }
    .addOnFailureListener { e ->
        Toast.makeText(this, "Error: ${e.message}",
Toast.LENGTH_SHORT).show()
    }
}

//FORGET USERNAME
forgotUsername.setOnClickListener {
    showEmailDialog("Recover Username") { email ->
        db.collection("users").whereEqualTo("email", email).get()
            .addOnSuccessListener {
documents ->
                if
(!documents.isEmpty) {
val username =
documents.documents[0].getString("username")
                AlertDialog.Builder(this)
                    .setTitle("Username Recovered")
                    .setMessage("Your username is: $username")
                    .setPositiveButton("OK", null)
                    .show()
            }
        }
    }
}

```

```

        } else {
            Toast.makeText(
this,
                "No username found for this email",
                Toast.LENGTH_SHORT
            ).show()
        }
    }
    .addOnFailureListener { e ->
        Toast.makeText(this, "Error: ${e.message}",
Toast.LENGTH_SHORT).show()
    }
}

//ADMIN ACCESS
adminAccess.setOnClickListener {
showAdminKeyDialog()
}
}

private fun showAdminKeyDialog() {
    val builder = android.app.AlertDialog.Builder(this)
    builder.setTitle("Enter Admin Key")

    val input = EditText(this)        input.hint = "Admin
Key"    input.inputType =
    android.text.InputType.TYPE_CLASS_TEXT or
    android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD

    builder.setView(input)

    builder.setPositiveButton("Enter") { _, _ ->

        val enteredKey = input.text.toString()
        val ref =
        FirebaseDatabase.getInstance().getReference("Admin/key")
        ref.get().addOnSuccessListener { snapshot ->

            val realKey =
            snapshot.value.toString().trim()                if
            (enteredKey == realKey) { //validating key
                startActivity(Intent(this,
AdminActivity::class.java))
            } else {
                Toast.makeText(this, "Wrong key",
Toast.LENGTH_SHORT).show()
            }

        }.addOnFailureListener {
            Toast.makeText(this, "Error connecting to server",
Toast.LENGTH_SHORT).show()
        }
    }
}

```

```
    }

    builder.setNegativeButton("Cancel", null)
builder.show()
}

private fun showEmailDialog(title: String, onConfirm: (String) -> Unit)
{
    val emailField = EditText(this)
    emailField.hint = "Enter your registered email"
}
```

```

        AlertDialog.Builder(this)
            .setTitle(title)
            .setView(emailField)
            .setPositiveButton("Submit") { _, _ ->
                val email =
                    emailField.text.toString().trim()
                if (email.isNotEmpty()) {
                    onConfirm(email)
                } else {
                    Toast.makeText(this, "Email cannot be empty",
                        Toast.LENGTH_SHORT).show()
                }
            }
            .setNegativeButton("Cancel", null)
            .show()
    }

    private fun loginWithEmail(email: String, password: String) {
        db.collection("users")
            .whereEqualTo("email", email).get()
            .addOnSuccessListener { documents ->

                if (!documents.isEmpty) {
                    val document = documents.documents[0]
                    val status = document.getString("status") ?: "active"

                    if (status == "disabled") {
                        auth.signOut()

                        Toast.makeText(
                            this,
                            "Account disabled by Admin. Please check back
                            later.",
                                Toast.LENGTH_SHORT
                            ).show()
                        return@addOnSuccessListener
                    }
                }

                auth.signInWithEmailAndPassword(email,
                    password)
                    .addOnCompleteListener(this) { task ->
                        if (task.isSuccessful) {
                            Toast.makeText(this, "Successfully Logged-In",
                                Toast.LENGTH_LONG)
                                    .show()

                            val intent = Intent(this,
                                HomeScreenActivity::class.java)
                            val managementCart = ManagmentCart(this)
                            managementCart.clearCart()
                        }
                    }
            }
    }

```

```
                startActivity(intent)

finish()
        } else {
            Toast.makeText(
this,
                "Invalid username or password",
                Toast.LENGTH_SHORT
            ).show()
        }
    }
    .addOnFailureListener {
        Toast.makeText(this, "Error checking account",
Toast.LENGTH_SHORT).show()
    }
} }
```

5.5.2 SELLER LISTINGS ACTIVITY CODE

```
package com.example.loginpage.Activity

import android.app.Activity import
    android.content.Intent import
    android.graphics.Bitmap import
    android.net.Uri import android.os.Bundle
    import android.provider.MediaStore
    import android.widget.* import
    androidx.appcompat.app.AppCompatActivity
    import
    com.example.loginpage.Domain.ItemsModel
import
    com.example.loginpage.databinding.ActivitySellerBinding
    import com.google.firebase.auth.FirebaseAuth import
    com.google.firebase.database.FirebaseDatabase import
    com.cloudinary.android.MediaManager
import com.cloudinary.android.callback.UploadCallback
//where seller is creating a listing class
    SellerListingsActivity : AppCompatActivity()
    {

        private lateinit var binding: ActivitySellerBinding

        private var imageUrl: Uri? = null
```

```

private val categoryNames = ArrayList<String>()
private val categoryIds = ArrayList<Int>()

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    binding = ActivitySellerBinding.inflate(layoutInflater)
    setContentView(binding.root)

    setupButtons()
    loadCategories()
}

// buttons
private fun setupButtons() {

    binding.btnUploadImage.setOnClickListener {
        val intent = Intent(Intent.ACTION_PICK)
        intent.type = "image/*"
        startActivityForResult(intent,
            100)
    }

    binding.btnCancel.setOnClickListener {
        clearForm()
        startActivity(Intent(this, SellerDashboardActivity::class.java))
        Toast.makeText(this, "Cancelled", Toast.LENGTH_SHORT).show()
    }

    binding.btnSubmit.setOnClickListener {
        uploadItem()
    }
}

private fun uploadItem() {

    val title = binding.etTitle.text.toString().trim()
    val desc = binding.etDescription.text.toString().trim()
    val priceText = binding.etPrice.text.toString().trim()

    if (title.isEmpty() || desc.isEmpty() || priceText.isEmpty() ||
        imageUri == null) {
        Toast.makeText(this, "Please fill all fields",
            Toast.LENGTH_SHORT).show()
        return
    }

    if (categoryIds.isEmpty()) {
        Toast.makeText(this, "Categories not loaded yet",
            Toast.LENGTH_SHORT).show()
        return
    }
}

```

```
    val selectedIndex = binding.spinnerCategory.selectedItemPosition
    val categoryId = categoryIds[selectedIndex]

    val price = priceText.toDoubleOrNull()
```

```

        if (price == null) {
            Toast.makeText(this, "Invalid price", Toast.LENGTH_SHORT).show()
        }
        return
    }

    val condition = getSelectedCondition()
    val userId = FirebaseAuth.getInstance().currentUser?.uid ?: ""

    val fileName = System.currentTimeMillis().toString()

    MediaManager.get().upload(imageUri)
        .unsigned("marketplaceApp")
        .callback(object : UploadCallback {
            override fun onStart(requestId: String?) {}
            override fun onProgress(requestId: String?, bytes: Long,
totalBytes: Long) {}
            override fun onSuccess(requestId:
String?, resultData: Map<*, *>?) {

                val imageUrl = resultData?.get("secure_url").toString()

                val itemId = FirebaseDatabase.getInstance()

                    .getReference("items")
                    .push()
                    .key!!

                val item = ItemsModel(
                    id = itemId,
                    title = title,
                    description = desc,
                    price = price,
                    picUrl = imageUrl,
                    condition = condition,
                    status = "pending",
                    userId = userId,
                    categoryId = categoryId,
                    isPopular = false,
                    timestamp =
                        System.currentTimeMillis()
                )

                FirebaseDatabase.getInstance()
                    .getReference("items")
                    .child(itemId)
                    .setValue(item)
                    .addOnSuccessListener {
                        Toast.makeText(this@SellerListingsActivity,
"Uploaded successfully", Toast.LENGTH_SHORT).show()
                        clearForm()
                        finish()
                    }
            }
        })
        .override fun
onError(requestId: String?, error:
com.cloudinary.android.callback.ErrorInfo?) {

```

```
Toast.makeText(this@SellerListingsActivity, "Upload  
failed",  
Toast.LENGTH_SHORT).show()  
}  
  
override fun onReschedule(requestId: String?, error:
```

```

com.cloudinary.android.callback.ErrorInfo?) {}
        })
        .dispatch()
    }

    // ----- CONDITION -----
    private fun getSelectedCondition(): String {

        return when
        (binding.radioCondition.checkedRadioButtonId) {
            binding.rbNew.id -> "New"           binding.rbLikeNew.id
            -> "Like New"           binding.rbUsed.id -> "Used"
        else -> "Unknown"
        }
    }

    // ----- CLEAR FORM -----
    private fun clearForm() {

        binding.etTitle.text.clear()
        binding.etDescription.text.clear()
        binding.etPrice.text.clear()
        binding.radioCondition.clearCheck()
        binding.spinnerCategory.setSelection(0)
        binding.imagePreview.setImageDrawable(null)
        imageUri = null
    }

    private fun loadCategories() {

        val ref = FirebaseDatabase.getInstance().getReference("Category")

        ref.get().addOnSuccessListener { snapshot ->

            categoryNames.clear()
            categoryIds.clear()

            for (child in snapshot.children) {

                val name = child.child("name").value.toString()
                val id = child.child("id").value.toString().toIntOrNull() ?: 0

                categoryNames.add(name)
                categoryIds.add(id)
            }

            val adapter = ArrayAdapter(
                this,
                android.R.layout.simple_spinner_dropdown_item,
                categoryNames
            )
        }
    }

```

```

        )

        binding.spinnerCategory.adapter = adapter
    }
}

private fun resizeImage(bitmap: Bitmap): Bitmap {
    val maxSize = 800        val ratio
    = minOf(
        maxSize.toFloat() /
        bitmap.width,
        maxSize.toFloat() / bitmap.height
    )

    val width = (bitmap.width * ratio).toInt()
    val height = (bitmap.height * ratio).toInt()

    return Bitmap.createScaledBitmap(bitmap, width, height, true)
}

// ----- IMAGE RESULT -----      override fun
onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
    super.onActivityResult(requestCode, resultCode, data)

    if (requestCode == 100 && resultCode == Activity.RESULT_OK) {

        imageUri = data?.data

        if (imageUri != null) {
            val bitmap =
MediaStore.Images.Media.getBitmap(contentResolver, imageUri)
            val resizedBitmap = resizeImage(bitmap)

            binding.imagePreview.setImageBitmap(resizedBitmap)
        }
    }
}
}
}

```

5.5.3 ADMIN ACTIVITY CODE

```
package com.example.loginpage.Activity

import
    android.content.Intent
    import
    android.os.Bundle import
    android.widget.TextView
    import
    android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.loginpage.databinding.ActivityAdminBinding
    import com.google.firebase.database.*
import com.google.firebase.firestore.FirebaseFirestore

class AdminActivity : AppCompatActivity() {
    private lateinit var databaseReference: DatabaseReference
    private lateinit var binding: ActivityAdminBinding

    private lateinit var db: FirebaseFirestore

    private lateinit var tvTotalItems: TextView
    private lateinit var tvPending: TextView
    private lateinit var tvApproved: TextView
    private lateinit var tvUsers: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityAdminBinding.inflate(layoutInflater)
        setContentView(binding.root)

        tvTotalItems = binding.tvTotalItems
        tvPending = binding.tvPending
        tvApproved = binding.tvApproved
        tvUsers = binding.tvUsers

        databaseReference =
        FirebaseDatabase.getInstance().getReference("items") db =
        FirebaseFirestore.getInstance()

        val cardPending =
        binding.cardPending val
        cardTotal = binding.cardTotal
        val cardUsers = binding.cardUsers

        //card views in Admin Dashboard
        cardPending.setOnClickListener {
            val intent = Intent(this, AdminPendingActivity::class.java)
            startActivity(intent)
        }
    }
}
```

```
cardTotal.setOnClickListener {  
    val intent = Intent(this, ItemTotalAdminActivity::class.java)  
    startActivity(intent)  
}  
  
cardUsers.setOnClickListener {  
    val intent = Intent(this,  
ViewAllUsersActivity::class.java)  
    startActivity(intent)    }
```

```

        loadDashboardData()
    setVariable()
    loadUserCount()
}

override fun onResume() {
    super.onResume()
    loadUserCount()
}

private fun loadDashboardData() {

    val ref = FirebaseDatabase.getInstance().getReference("items")

    ref.addValueEventListener(object : ValueEventListener {

        override fun onDataChange(snapshot: DataSnapshot) {

            var pending = 0
            var approved = 0
            var rejected = 0

            for (child in snapshot.children) {
                val status =
                    child.child("status").getValue(String::class.java)

                when (status) {
                    "approved" -> approved++ //counting items in the
marketplace database
                    "pending" -> pending++
                    "rejected" -> rejected++
                }
            }

            tvPending.text = pending.toString()
            tvApproved.text = approved.toString()
            tvTotalItems.text = (approved + pending + rejected).toString()
        }

        override fun onCancelled(error: DatabaseError) {}
    })
}

//logout button
private fun setVariable() {
    binding.adLogout.setOnClickListener {

        com.google.firebase.auth.FirebaseAuth.getInstance().signOut()
    }
}

```

```
        val intent = Intent(this,
LoginActivity::class.java)           intent.flags =
Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
startActivity(intent)
    }
}

private fun loadUserCount() {
db.collection("users")
    .addSnapshotListener { documents, error
->          if (error != null || documents
== null) {
        Toast.makeText(this, "Failed to load users",
Toast.LENGTH_SHORT).show()
        return@addSnapshotListener
    }

        tvUsers.text =
documents.size().toString()    }
}
}
```

5.6 DATABASE TABLES

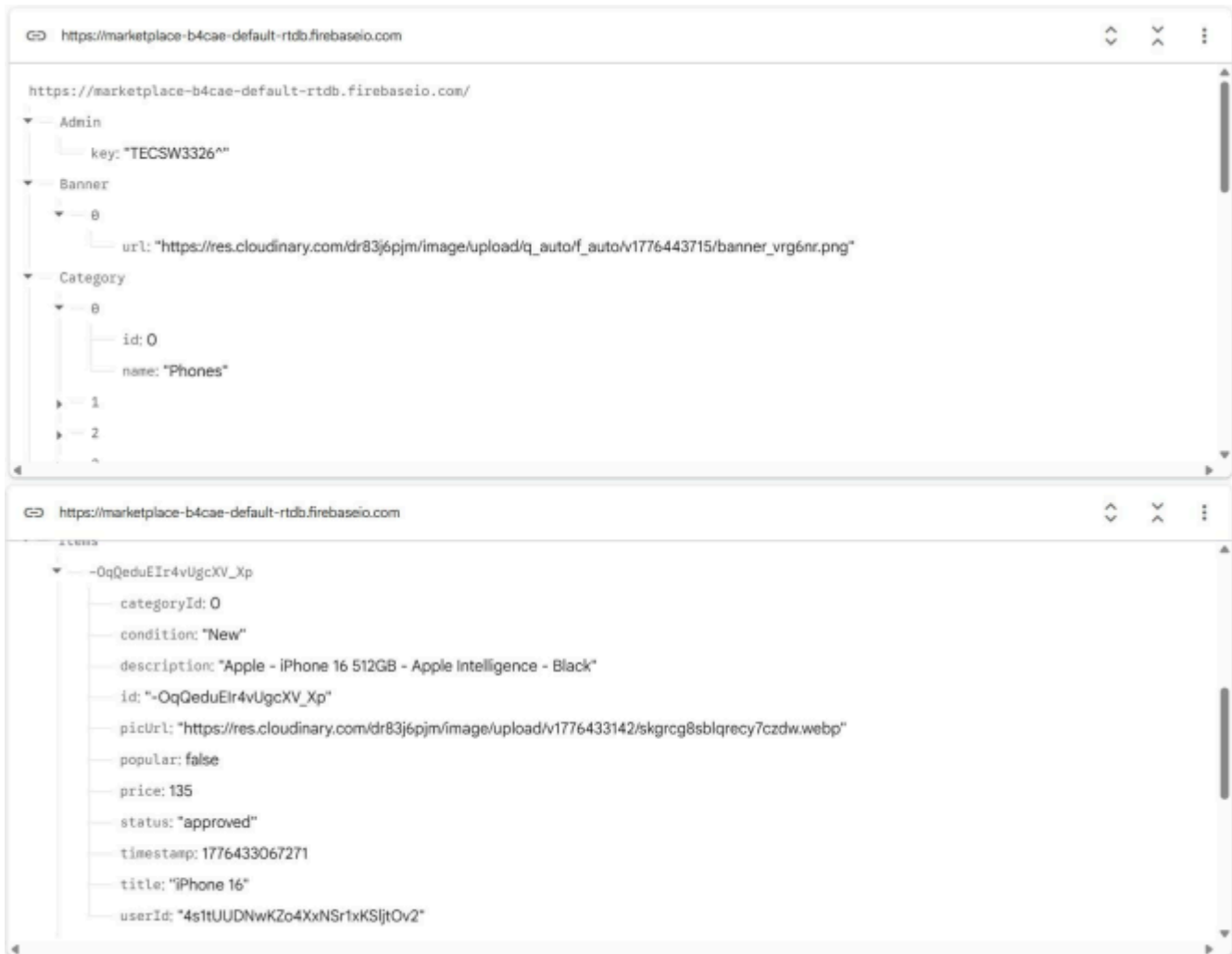
5.6.1 USER DATABASE: FIRESTORE

More in Google Cloud ▼ Panel view Query builder ⋮

🏠 > users > amy455

🏠 (default)	📁 users ≡ ⋮	📄 amy455 ⋮
+ Start collection	+ Add document	+ Start collection
users >	amy455 >	+ Add field
	amy55 amyp45 ayesha22 k8dith mabel89	email: "mabel2@gmail.com" name: "Mabel P" status: "active" username: "amy455"

5.6.2 ITEMS AND CATEGORY: REALTIME DATABASE



6. Assumptions and Constraints

6.1 ASSUMPTIONS

- TechSwap is mainly made for students who want to buy and sell electronic items in a secure environment.
- For the Login and Registration feature, the admins of the app will already have premade credentials for us to be able to look at listings and manage the database of the app.
- Users logging into the app for the first time will provide proper credentials for both the username and password fields
- A user can be both a buyer and a seller, and admin accounts are handled separately
- For the payment process of this app, it will not go through a real payment system. No real financial transactions will occur
- For both the seller and buyers, all the required fields (product name, brand/model, price, condition and photos), will be and must be filled in if they are going to show up as a listing in this app
- All listings submitted by a seller are subject to approval, and the admins are responsible for approving all listings
- For the database structure of this app, storing the users, listings, orders, and cart information will be stored locally
- Admins can look at the user database to make sure that all listings are updated for both the buyers and sellers

6.2 CONSTRAINTS

- App must be completed in time for presentations in April, along with videos and documentation
- This app has to function like a marketplace app, handled by buyers, sellers, and the admins
- No real payment system for checkout of items
- The app will be developed as an android app that will be able to run on majority of android models, older models may not be able to support the app
- Must provide different documentation as we develop our app and documentation must align with the features in our app
- The application will only handle basic error handling, like authentication of Login credentials, listing details, and checkout information

6.3 OUT OF SCOPE MATERIAL

- Real time messaging between the buyers and sellers
- Ratings and reviews on sellers and their items
- Real transaction system between the seller, buyers, and the app
- Tracking and shipping status for the buyers

7. Delivery and Schedule

List all tasks/milestones from start of the project to the end with specific dates for both Anticipated Start & End Dates.

Task/Milestone Description	Anticipated Start Date	Anticipated End Date	Status	Comments
Prepare Requirements and UML diagram	Jan 28, 2026	Feb 13, 2026	Completed	• Project requirements have been set. • UML diagrams have been created.
SRA document (Includes project objectives, Requirements and UML diagrams)	Feb 13, 2026	March 2, 2026	Completed	• The SRA document is in the process of being created. • Is set to be completed by March 2.
Test case design	March 3, 2026	April 11, 2026	Completed	• Begins with Test Plan template is distributed. • Ends with Test Plan document due with Increment 3 on Apr 11.
External Documentation (i.e. User Manual)	March 20, 2026	April 22, 2026	Completed	• Draft during later development • Finalize before presentations that start Apr 13-22.
Project presentation	April 13, 2026	April 22, 2026	Completed	• The presentation window in schedule is April 13-22 • Start prep a bit earlier that week.
Final Milestone: project delivery	April 20, 2026	April 24, 2026	Completed	• Aligns with Increment 4 / Final Project • Due Apr 24 (project binder and evaluation forms)

8. Stakeholder Approval Form

Stakeholder Name	Stakeholder Role	Stakeholder Comments	Stakeholder Approval Signature and Date
Professor Bhanu Jain	Project Manager		
Aman Gulati	Asst. Project Manager		
Islam Shaikh	Asst. Project Manager		
Kate Lopez	Developer		
Mabel Pokoo	Developer		
Ayesha Baig	Developer		
Salima Salman	Developer		

Appendix:

None